

# 2-WavelengthCalibration

April 4, 2025

## 0.1 Astronomía Observacional

## 0.2 Astropy, Tutorial 2: Reducción de datos espectroscópicos, parte 2: calibración de longitud de onda

María Alejandra Moreno, Cesar Mauricio Mendoza

Este tutorial explicará la extracción del espectro de una lámpara de calibración a partir de una traza existente.

A continuación, se demostrará la identificación de líneas mediante la base de datos de listas de líneas del NIST.

Por último, se mostrará cómo ajustar una solución de longitud de onda a un espectro de calibración, integrando la información de múltiples lámparas de calibración.

```
[50]: from PIL import Image as PILImage
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['image.origin'] = 'lower'
plt.style.use('dark_background')
```

Queremos mostrar imágenes, no matrices, por lo que establecemos el origen en la esquina inferior izquierda. Configuración opcional: si se ejecuta, se verá bien sobre un fondo oscuro.

```
[51]: from astropy import units as u
from astropy.modeling.polynomial import Polynomial1D
from astropy.modeling.models import Gaussian1D, Linear1D
from astropy.modeling.fitting import LinearLSQFitter
from IPython.display import Image
# astroquery provides an interface to the NIST atomic line database
from astroquery.nist import Nist
```

Disponemos de tres espectros de lámparas de calibración: mercurio, criptón y neón. Estos se guardan como archivos .bmp (mapa de bits).

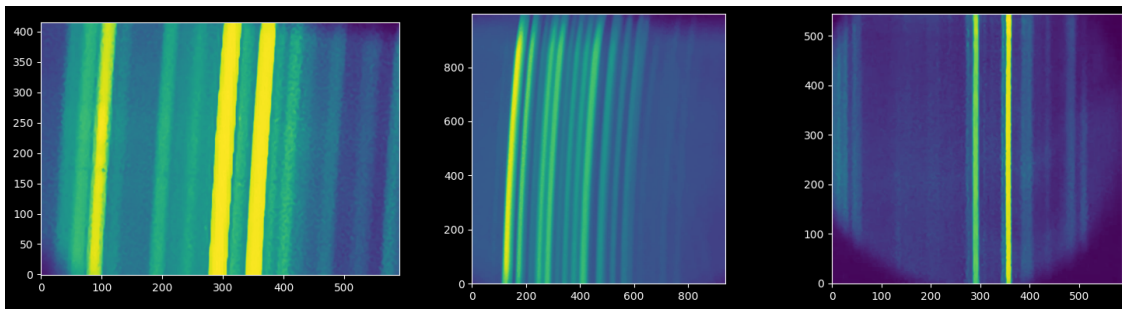
```
[52]: hg_filename = "hg_lamp_1-sixteenth_s.bmp"
kr_filename = "kr_lamp_p6.bmp"
ne_filename = "ne_lamp_1s.bmp"
kr_filenameu = "Krypton.jpeg"
```

```
hg_filenameu = "hg_lamp_lab.bmp"
ne_filenameu = "Neon.jpeg"
```

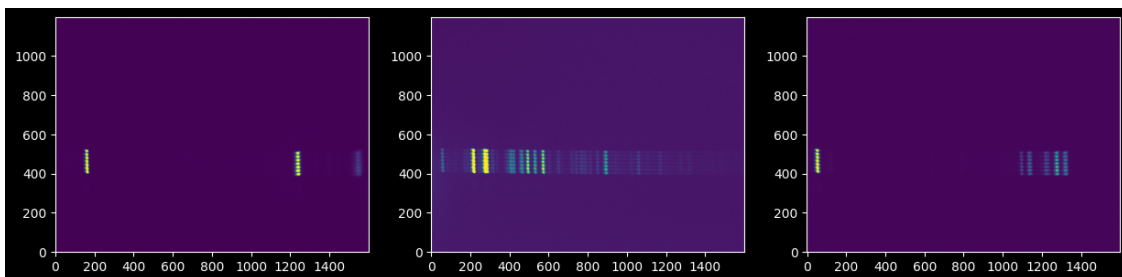
```
[53]: hg_image = np.array(PILImage.open(hg_filename))
      kr_image = np.array(PILImage.open(kr_filename))
      ne_image = np.array(PILImage.open(ne_filename))

      kr_imageu = np.array(PILImage.open(kr_filenameu).convert("L"))
      hg_imageu = np.array(PILImage.open(hg_filenameu).convert("L"))
      ne_imageu = np.array(PILImage.open(ne_filenameu).convert("L"))
```

```
[54]: fig, (hg, ne, kr) = plt.subplots(1, 3, figsize=(15, 4))
      hg.imshow(hg_imageu);
      kr.imshow(kr_imageu);
      ne.imshow(ne_imageu);
      plt.tight_layout()
      plt.show()
```



```
[55]: fig, (hgs, nes, krs) = plt.subplots(1, 3, figsize=(15, 4))
      krs.imshow(kr_image);
      nes.imshow(ne_image);
      hgs.imshow(hg_image);
```



Podemos observar al ampliar la imagen que parece haber siete espectros independientes. EN esta

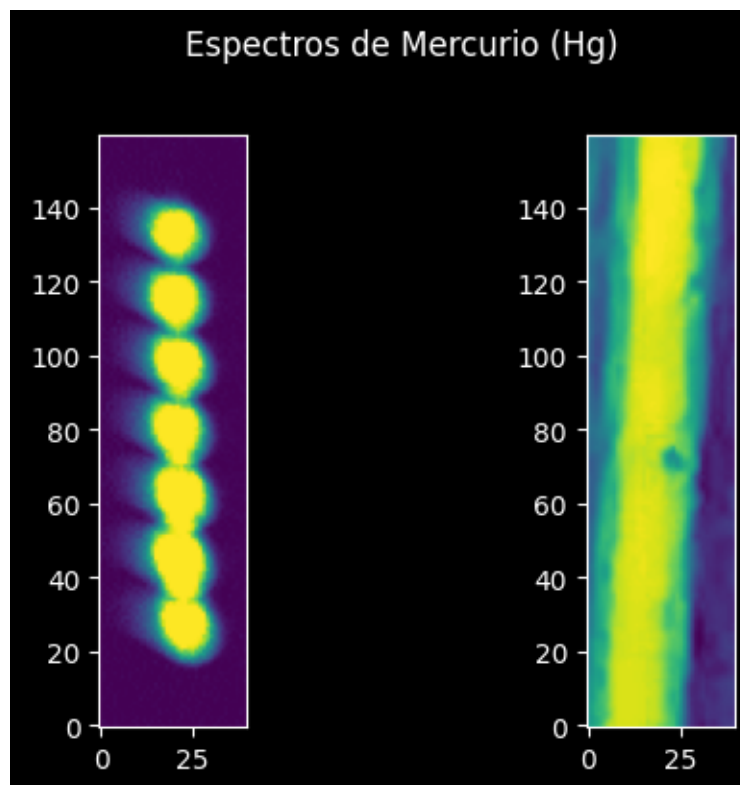
caso usamos las lineas experimentales de hg y la tomada del espectro del ejemplo.

```
[56]: fig, (s, o) = plt.subplots(1,2 , figsize=(6, 4))
s.imshow(hg_image[380:540,140:180]);
o.imshow(hg_imageu[100:260 , 80:120])
fig.suptitle("Espectros de Mercurio (Hg)", y=1.02)

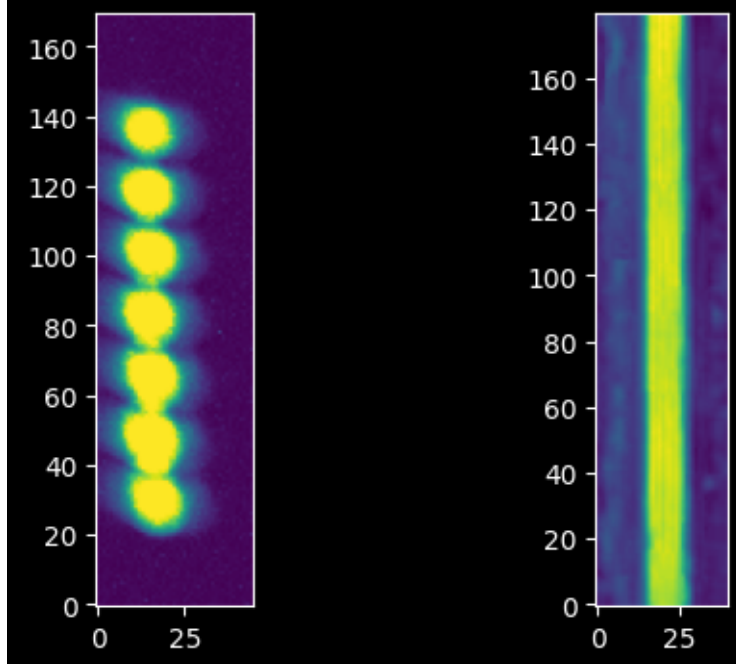
fig2, (s2, o2) = plt.subplots(1,2 , figsize=(6, 4))
s2.imshow(kr_image[380:550,40:85]);
o2.imshow(kr_imageu[0:180 , 270:310])
fig2.suptitle("Espectros de Kripton (Kr)", y=1.02)

fig3, (s3, o3) = plt.subplots(1,2 , figsize=(6, 4))
s3.imshow(ne_image[380:550,200:240]);
o3.imshow(ne_imageu[0:180 , 270:310])
fig3.suptitle("Espectros de Neon (Ne)", y=1.02)
```

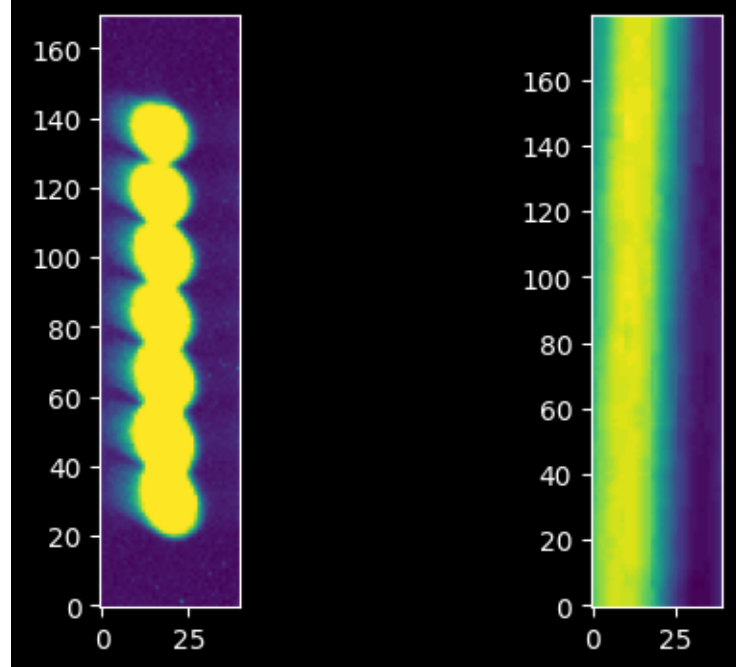
```
[56]: Text(0.5, 1.02, 'Espectros de Neon (Ne)')
```



### Espectros de Kripton (Kr)



### Espectros de Neon (Ne)



Recreamos nuestro modelo de traza del Tutorial de Traza Espectroscópica utilizando los modelos ajustados.

(Podríamos haber utilizado la traza empírica directamente, lo que podría resultar en características de ruido ligeramente mejoradas, pero para simplificar y para que ambos cuadernos se puedan utilizar de forma independiente, utilizamos aquí los modelos polinomiales y gaussianos ajustados).

```
[57]: linfitter = LinearLSQFitter()
```

```
[136]: # Para hg_image con constantes ajustadas.
trace_model = Polynomial1D(degree=3, c0=453.6307, c1=-0.01596396, c2=0.
↳0000008259, c3=3.3348554642250357e-09)
trace_profile_model = Gaussian1D(amplitude=123.84846797, mean=0.17719819,
↳stddev=5.10872134)
xaxis = np.arange(hg_image.shape[1])
trace_center = trace_model(xaxis)
npixels_to_cut=15
yaxis = np.arange(-npixels_to_cut, npixels_to_cut)
model_trace_profile = trace_profile_model(yaxis)

#Para kr_image

xvals1= np.arange(kr_image.shape[1])
yaxis3 = np.repeat(np.arange(400, 520)[: ,None], kr_image.shape[1], axis=1)
weighted_yaxis_valueskr = np.average(yaxis3, axis=0, weights=kr_image[400:520,:
↳] - np.median(kr_image))
polymodelkr = Polynomial1D(degree=3)
fitted_polymodel3 = linfitter(polymodelkr, xvals1, weighted_yaxis_valueskr)
trace_centerkr_image = fitted_polymodel3(xvals1)

#Para ne_image

xvals2 = np.arange(ne_image.shape[1])
yaxis4 = np.repeat(np.arange(420, 500)[: ,None], ne_image.shape[1], axis=1)
weighted_yaxis_valuesne = np.average(yaxis4, axis=0, weights= ne_image[420:500,
↳:] - np.median(ne_image))
polymodelne = Polynomial1D(degree=3)
fitted_polymodel4 = linfitter(polymodelne, xvals2, weighted_yaxis_valuesne)
trace_centerne_image = fitted_polymodel4(xvals2)

#para kr_imageu
```

```

trace_modelkr = Polynomial1D(degree=3, c0=186, c1=-0.01596396, c2=0.0000008259,
↪c3=3.3348554642250357e-09)
trace_profile_modelkr = Gaussian1D(amplitude=186, mean=0.17719819, stddev=5.
↪10872134)
xvals5 = np.arange(kr_imageu.shape[1])
trace_centerkr = trace_modelkr(xvals5)
npixels_to_cut=15
yaxis5 = np.arange(-npixels_to_cut, npixels_to_cut)
model_trace_profilekr = trace_profile_modelkr(yaxis5 )

#para ne_imageu

trace_modelne = Polynomial1D(degree=3, c0=412.26, c1=-0.01596396, c2=0.
↪0000008259, c3=3.3348554642250357e-09)
trace_profile_modelne = Gaussian1D(amplitude=412.26, mean=0.17719819, stddev=5.
↪10872134)
xvals6 = np.arange(ne_imageu.shape[1])
trace_centerne = trace_modelne(xvals6)
npixels_to_cut=15
yaxis6 = np.arange(-npixels_to_cut, npixels_to_cut)
model_trace_profilene = trace_profile_modelne(yaxis6 )

#para hg_imageu

trace_modelhg = Polynomial1D(degree=3, c0=200, c1=-0.01596396, c2=0.0000008259,
↪c3=3.3348554642250357e-09)
trace_profile_modelhg = Gaussian1D(amplitude=200, mean=0.17719819, stddev=5.
↪10872134)
xaxishg = np.arange(hg_imageu.shape[1])
trace_centerhg = trace_modelhg(xaxishg)
npixels_to_cut=15
yaxishg = np.arange(-npixels_to_cut, npixels_to_cut)
model_trace_profilehg = trace_profile_modelhg(yaxishg)

fitted_polymodel6

```

[136]: <Polynomial1D(3, c0=412.26456193, c1=-0.01823617, c2=0.00003638, c3=-0.00000002)>

Para las firmas que se tomaron en el laboratorio se debe hacer un ajuste polinomicos desde sin asumir las constante

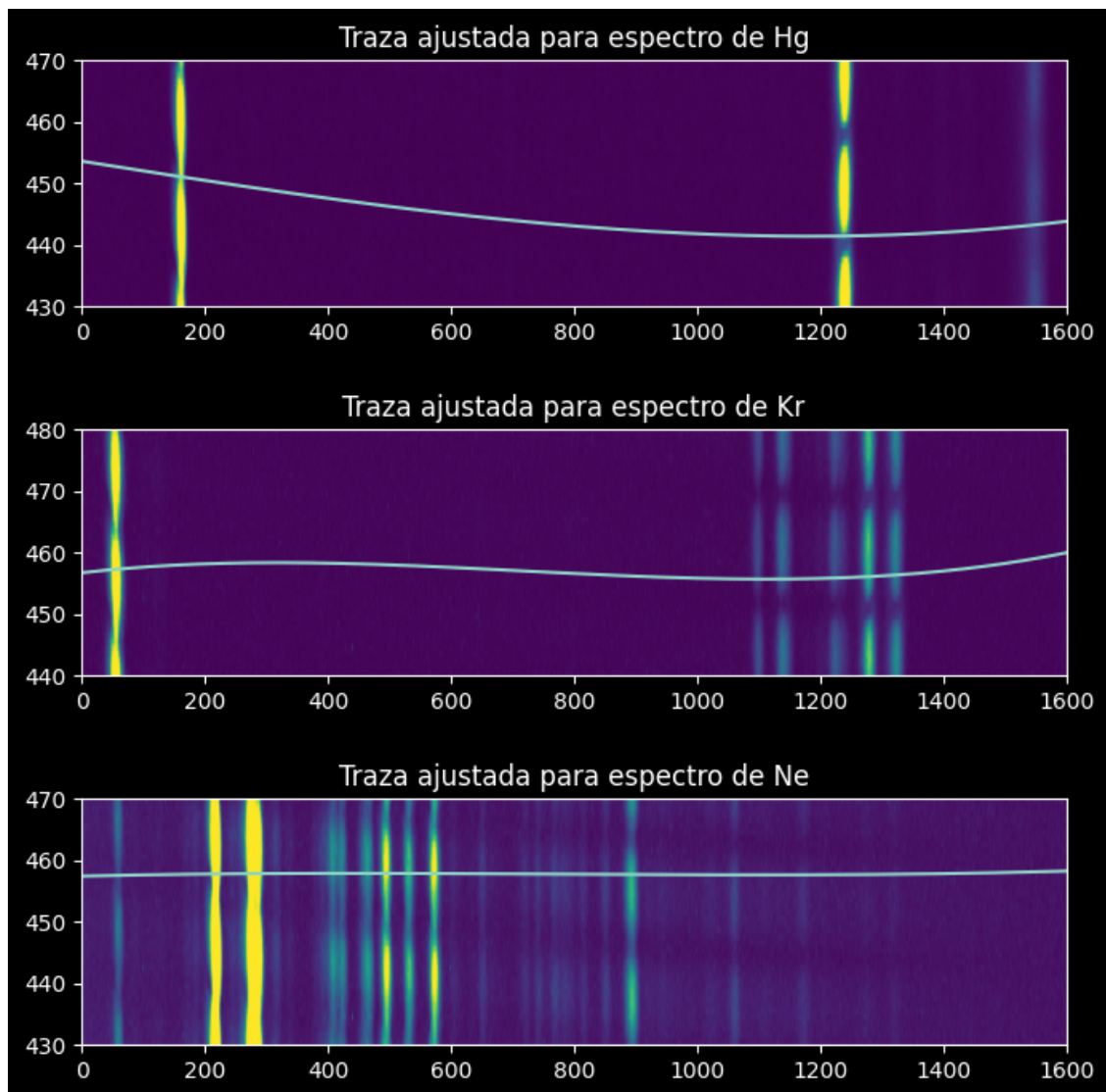
Luego, utilizamos la traza para extraer los espectros de cada una de las tres lámparas de calibración.

Primero verificamos que las trazas tengan un aspecto aceptable:

```
[59]: plt.figure(figsize=(16,8))
ax1 = plt.subplot(3,1,1)
ax1.imshow(hg_image[430:470,:], extent=[0,1600,430,470])
ax1.plot(xaxis, trace_center)
ax1.set_title("Traza ajustada para espectro de Hg")

ax1.set_aspect(10)
ax2 = plt.subplot(3,1,2)
ax2.imshow(kr_image[430:470,:], extent=[0,1600,440,480])
ax2.plot(xvals1, trace_centerkr_image)
ax2.set_aspect(10)
ax2.set_title("Traza ajustada para espectro de Kr")

ax3 = plt.subplot(3,1,3)
ax3.imshow(ne_image[430:470,:], extent=[0,1600,430,470])
ax3.plot(xvals2, trace_centerne_image)
ax3.set_aspect(10)
ax3.set_title("Traza ajustada para espectro de Ne")
plt.subplots_adjust(hspace=0.5)
```



```
[138]: plt.figure(figsize=(16, 8))

# Primer subgráfico
ax1 = plt.subplot(3, 1, 1) # (2 filas, 1 columna, primer subplot)
ax1.imshow(kr_imageu[170:200, :], extent=[0, 600, 170, 200])
ax1.plot(xvals5, trace_centerkr[:len(xvals5)])
ax1.set_aspect(10)
ax1.set_title("Traza ajustada para espectro de Kr")

# Segundo subgráfico
ax2 = plt.subplot(3, 1, 2) # (2 filas, 1 columna, segundo subplot)
ax2.imshow(ne_imageu[390:430, :], extent=[0, 950, 390, 430])
ax2.plot(xvals6, trace_centerne[:len(xvals6)])
```

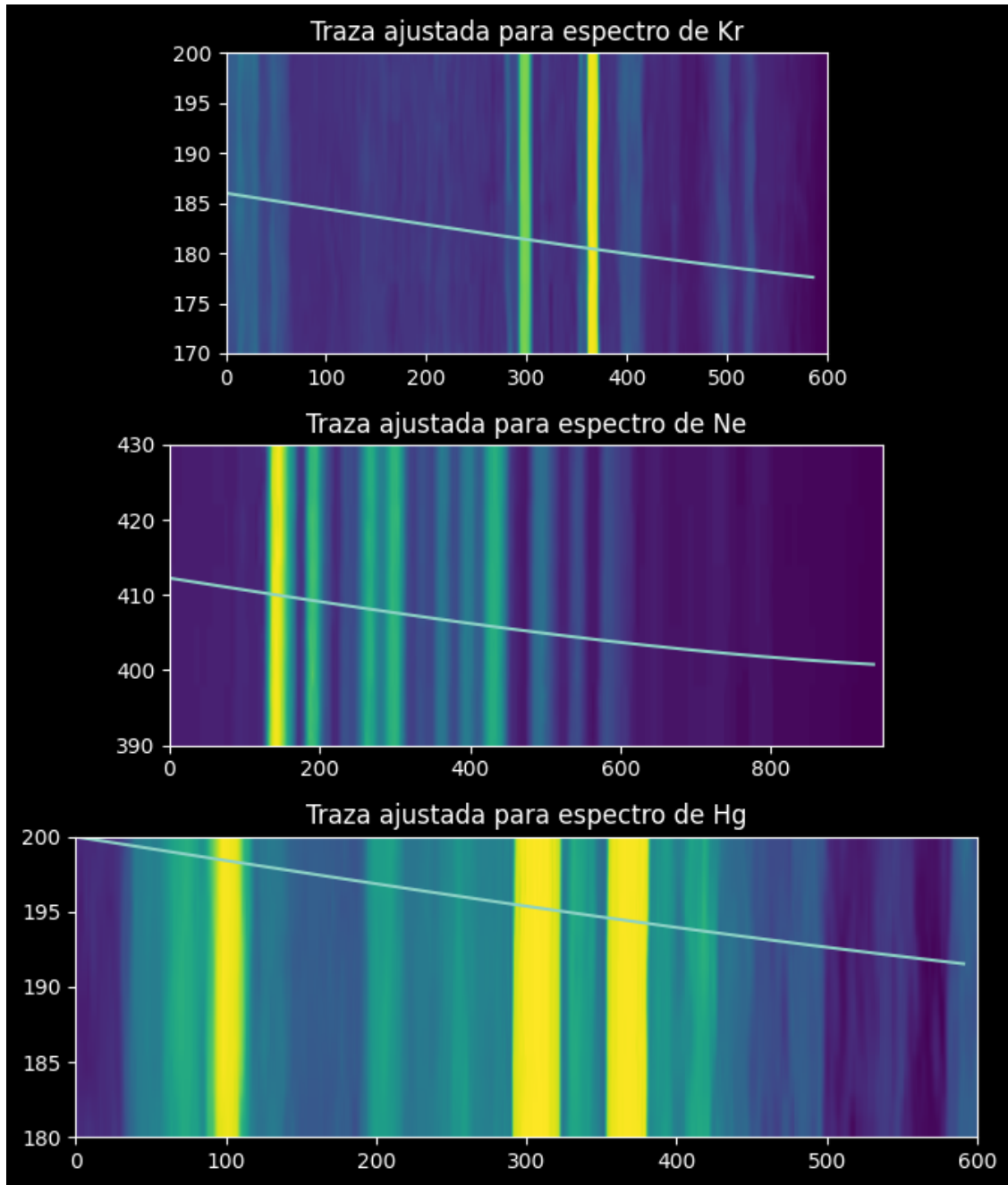


```

ax2.set_aspect(10)
ax2.set_title("Traza ajustada para espectro de Ne")

# Tercer subgráfico
ax3 = plt.subplot(3, 1, 3) # (2 filas, 1 columna, segundo subplot)
ax3.imshow(hg_imageu[220: 235, :], extent=[0, 600, 180, 200])
ax3.plot(xaxishg, trace_centerhg[:len(xaxishg)])
ax3.set_aspect(10)
ax3.set_title("Traza ajustada para espectro de Hg")
plt.tight_layout()

```



En las celdas siguientes, extraemos el espectro promedio ponderado por perfil de traza de cada una de las tres lámparas de calibración.

`npixels_to_cut` define una región de recorte alrededor de la traza, similar a la que se muestra en las figuras anteriores. El `model_trace_profile`, obtenido anteriormente, es el perfil de transmisión que medimos utilizando la traza estelar.

```
[61]: hg_spectrum = np.array([np.average(hg_image[int(yval)-npixels_to_cut:
    ↪int(yval)+npixels_to_cut, ii],
                                weights=model_trace_profile)
    for yval, ii in zip(trace_center, xaxis)])
```

```
[62]: ne_spectrum = np.array([np.average(ne_image[int(yval)-npixels_to_cut:
    ↪int(yval)+npixels_to_cut, ii],
                                weights=model_trace_profile)
    for yval, ii in zip(trace_center, xaxis)])
```

```
[63]: kr_spectrum = np.array([np.average(kr_image[int(yval)-npixels_to_cut:
    ↪int(yval)+npixels_to_cut, ii],
                                weights=model_trace_profile)
    for yval, ii in zip(trace_center, xaxis)])
```

```
[120]: kru_spectrum = np.array([np.average(kr_imageu[int(yaxis5)-npixels_to_cut:
    ↪int(yaxis5)+npixels_to_cut, ii],
                                weights=model_trace_profilekr)
    for yaxis5, ii in zip(trace_centerkr, xvals5)])
```

```
[140]: neu_spectrum = np.array([np.average(ne_imageu[int(yaxis6)-npixels_to_cut:
    ↪int(yaxis6)+npixels_to_cut, ii],
                                weights=model_trace_profilene)
    for yaxis6, ii in zip(trace_centerne, xvals6)])
```

```
[69]: hgu_spectrum = np.array([np.average(hg_imageu[int(yaxishg)-npixels_to_cut:
    ↪int(yaxishg)+npixels_to_cut, ii],
                                weights=model_trace_profilehg)
    for yaxishg, ii in zip(trace_centerhg, xaxishg)])
```

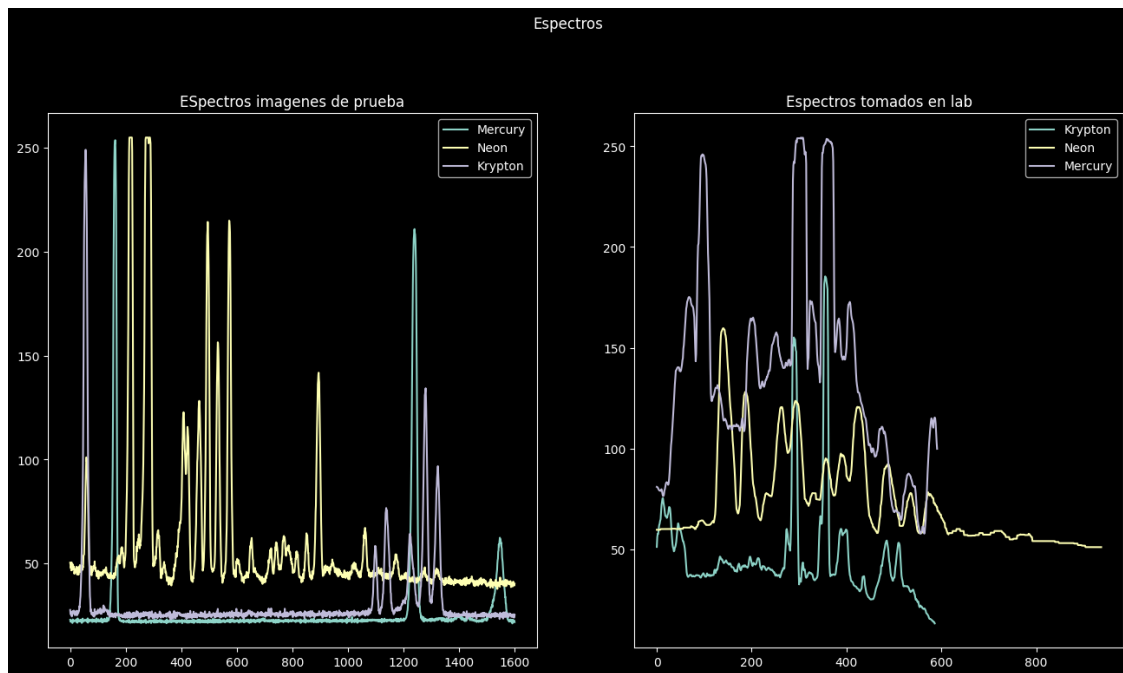
Graficamos para obtener los espectros con unidades despreciables

```
[141]: fig, (s, o) = plt.subplots(1,2, figsize=(16, 8))
s.plot(xaxis, hg_spectrum, label='Mercury');
s.plot(xaxis, ne_spectrum, label='Neon')
s.plot(xaxis, kr_spectrum, label='Krypton')
s.legend()
s.set_title("ESpectros imagenes de prueba")

o.plot(xvals5, kru_spectrum, label='Krypton')
o.plot(xvals6, neu_spectrum, label='Neon')
o.plot(xaxishg, hgu_spectrum, label='Mercury')
o.legend()
o.set_title("Espectros tomados en lab")
```

```
fig.suptitle("Espectros", y=1.02)
```

```
[141]: Text(0.5, 1.02, 'Espectros')
```



Ahora tenemos los espectros de una lámpara de mercurio, una de neón y una de criptón.

Si no tenemos conocimientos previos, tendríamos que hacer conjeturas y comprobaciones. Sin embargo, es útil hacer conjeturas y comprobaciones fundamentadas; Wikipedia es un buen recurso para indicarnos las listas de líneas correctas:

- [https://en.wikipedia.org/wiki/Mercury-vapor\\_lamp#Emission\\_line\\_spectrum](https://en.wikipedia.org/wiki/Mercury-vapor_lamp#Emission_line_spectrum)
- [https://en.wikipedia.org/wiki/Gas-discharge\\_lamp#Color](https://en.wikipedia.org/wiki/Gas-discharge_lamp#Color)

```
[ ]: Image("https://upload.wikimedia.org/wikipedia/commons/2/29/Mercury_Spectra.jpg")
```



```
[ ]: Image("https://upload.wikimedia.org/wikipedia/commons/9/99/Neon_spectra.jpg")
```

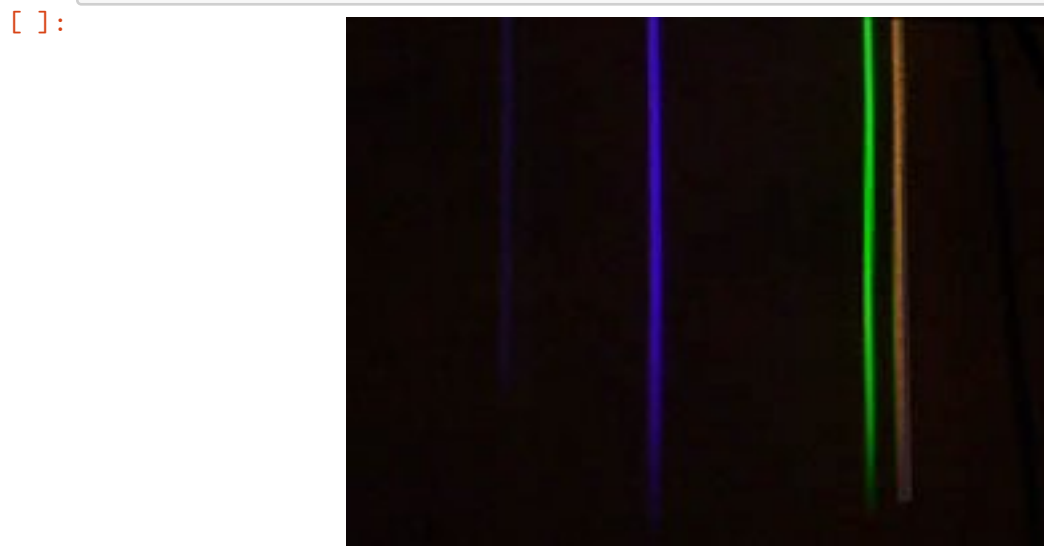


```
[ ]: Image("https://upload.wikimedia.org/wikipedia/commons/a/a6/Krypton_Spectrum.  
↪jpg")
```



Comencemos con el mercurio. Usamos un espectro de Wikipedia diferente, un poco más parecido al nuestro, para guiarnos visualmente (tenga en cuenta que las relaciones de las líneas varían de una lámpara a otra, al menos en parte porque cada lámpara contiene gas a una presión diferente).

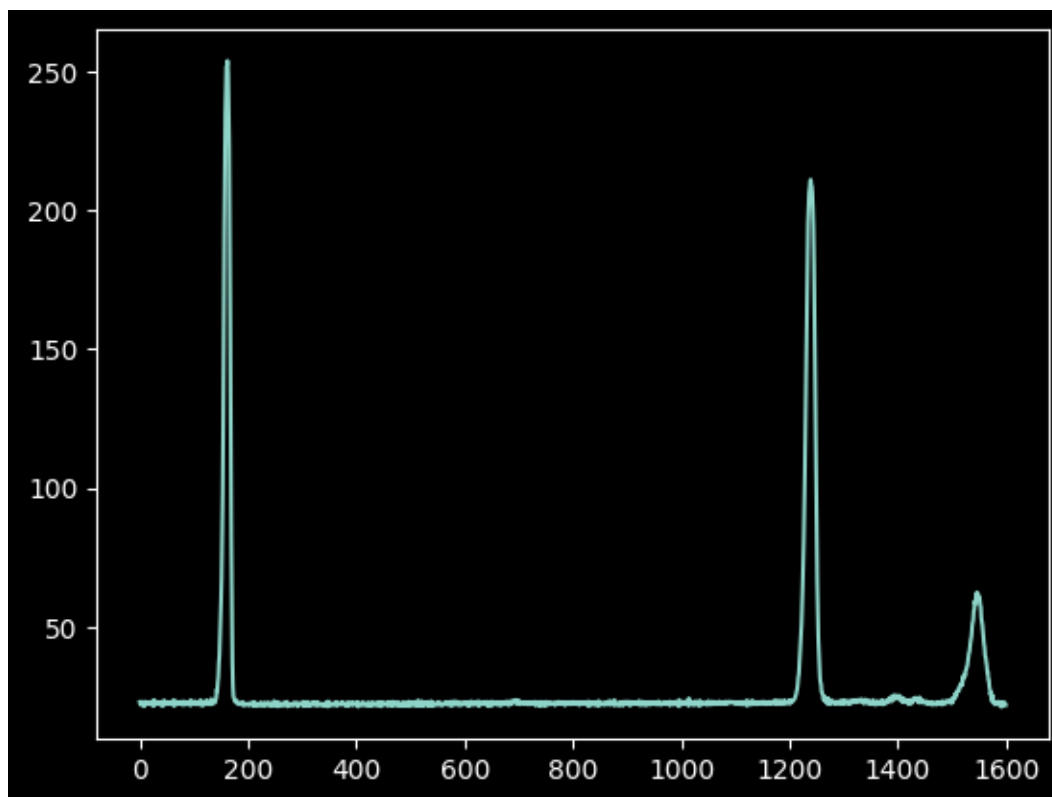
```
[ ]: Image("https://upload.wikimedia.org/wikipedia/commons/0/0d/HG-Spektrum_crop.  
↪jpg")
```



Las líneas ópticas de Mercurio, según Wikipedia, se expresan en ángstroms:

- 4047 violeta
- 4358 azul
- 5461 verde
- 5782 amarillo anaranjado
- 6500 rojo

```
[ ]: plt.plot(xaxis, hg_spectrum);
```



En el siguiente paso, debemos anotar qué valores de píxel (aproximados) corresponden a longitudes de onda conocidas.

Adivinar estos valores simplemente observando los espectros y comparándolos con atlas espectrales es difícil y requiere experiencia.

Si tienes espectros tomados con la misma configuración de una fuente conocida, como una estrella A0, podrías usarlos para obtener una “previa” sobre dónde se centra el espectro. En este caso, nuestro conjunto de espectros incluye Deneb, que es una estrella A. Tiene una línea de absorción clara (que veremos más adelante, en el tutorial 3) alrededor del píxel 750. Las estrellas A tienen líneas de absorción de hidrógeno profundas en su atmósfera y poco más, por lo que probablemente se trate de una línea de hidrógeno. Podemos asumir que es H-alfa o H-beta, ya que ambas están en el campo óptico y son intensas (las líneas superiores, como H-delta, están muy juntas, por lo que si fuera una de ellas, esperaríamos ver varias líneas adyacentes).

Por lo tanto, podemos comparar nuestro espectro con los atlas, suponiendo que el píxel 750 es H-beta o H-alfa. Usando la lista de líneas, podemos ver que el rojo es bastante improbable: H-alfa está en 6563, por lo que habría una línea justo al lado. Si es H-beta, hay líneas a ambos lados y hay un doblete a la derecha.

El siguiente truco es averiguar en qué dirección aumenta la longitud de onda: ¿hacia la izquierda o hacia la derecha? Mercurio no ayuda con eso, ya que los dobletes azul-violeta y verde-amarillo están aproximadamente a la misma distancia entre sí. Resulta, a partir de un trabajo de conjeturas y comprobaciones, que la longitud de onda aumenta hacia la izquierda.

De nuevo, esta parte es difícil de hacer directamente a partir de los datos; generalmente, se espera tener una idea bastante precisa de la longitud de onda que se está observando antes de observar.

```
[93]: guessed_wavelengths = [546.1, 435.8, 404.7]
      guessed_xvals = [165, 1230, 1550]
```

Para nuestros espectros de laboratorio

---

|                                  |
|----------------------------------|
| l cripton presenta               |
| 427.4 nm (azul-violeta)          |
| 431.9 nm (azul-violeta)          |
| - 557.0 nm (verde)               |
| 587.1 nm (amarillo, muy intensa) |
| 605.7 nm (naranja)               |
| 645.6 nm (rojo)                  |

---

El neon presenta - 540.1 nm (verde)

- 585.2 nm (amarillo-naranja)
- 588.2 nm (amarillo-naranja)
- 603.0 nm (naranja)
- 607.4 nm (naranja)
- 616.3 nm (rojo)
- 621.7 nm (rojo)
- 626.6 nm (rojo)
- 633.4 nm (rojo intenso, una de las más brillantes)
- 638.3 nm (rojo)
- 640.2 nm (rojo)
- 650.6 nm (rojo)
- 659.9 nm (rojo)
- 692.9 nm (rojo profundo)
- 703.2 nm (rojo profundo, una de las más intensas)

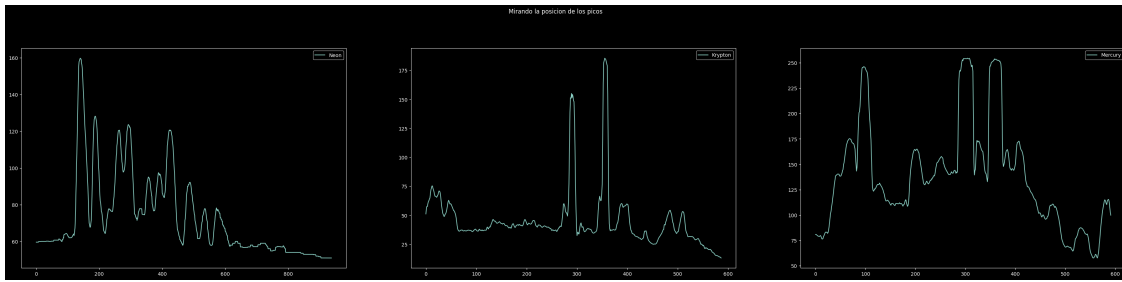
```
[144]: guessed_wavelengthsne = [ 588.2 , 603.0, 607.4 , 616.3 , 621.7 , 626.6 , 633.4,
↪, 638.3 , 640.2, 650.6]
      guessed_wavelengthskr = [431.9 , 557.0 , 605.7 , 645.6]
      guessed_wavelengthshg = [404.7, 435.8, 546.1, 578.2, 650.0]
```

```
[145]: fig, (s, o , a) = plt.subplots(1,3, figsize=(40, 8))
      s.plot(xvals6, neu_spectrum, label='Neon');
      s.legend()
```

```
o.plot(xvals5, kru_spectrum, label='Krypton')
o.legend()

a.plot(xaxishg, hgu_spectrum, label= 'Mercury')
a.legend()
fig.suptitle("Mirando la posicion de los picos", y=1.02)
```

[145]: Text(0.5, 1.02, 'Mirando la posicion de los picos')



```
[146]: guessed_xvalskr = [ 290 , 350,400, 520]
guessed_xvalsne = [175, 200, 250, 300, 350,390,430, 500, 550, 600]
guessed_xvalshg = [98, 200, 310, 350, 590]
```

### 0.3 Mejorando nuestras suposiciones

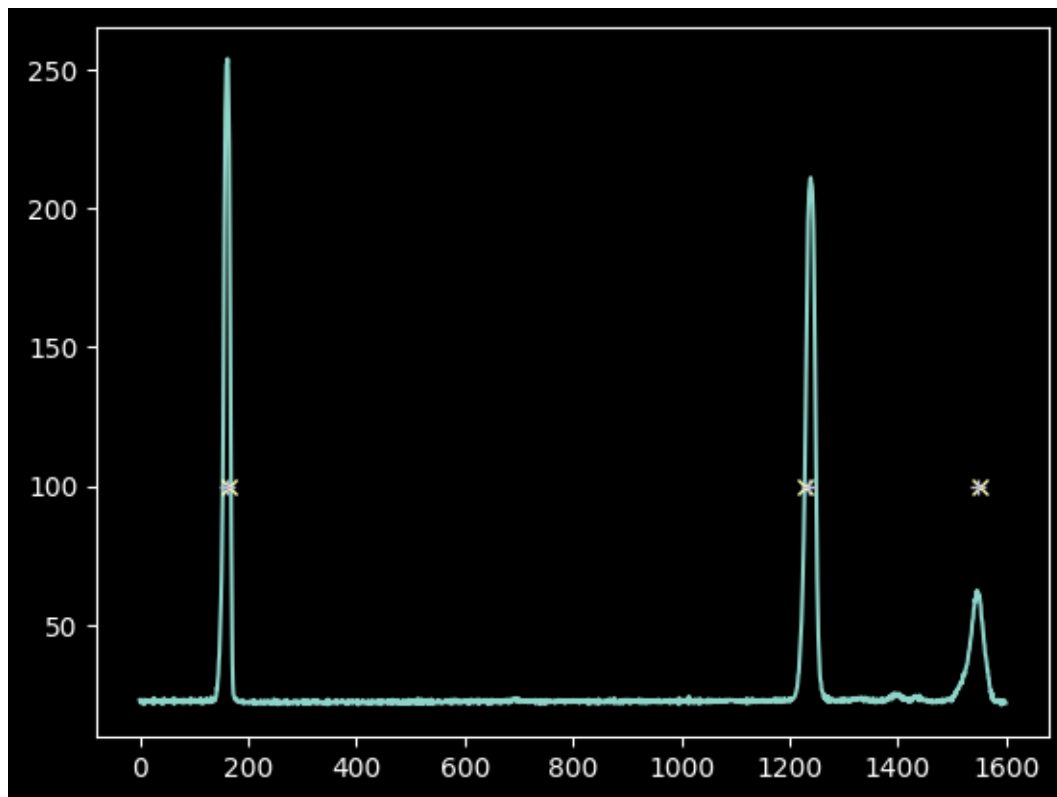
Podemos determinar mucho mejor los valores X de los píxeles tomando la coordenada ponderada por la intensidad (momento 1):

```
[147]: npixels = 15
improved_xval_guesses = [np.average(xaxis[g-npixels:g+npixels],
                                weights=hg_spectrum[g-npixels:g+npixels] -
                                np.median(hg_spectrum))
                        for g in guessed_xvals]
improved_xval_guesses
```

```
[147]: [np.float64(160.736647147187),
np.float64(1235.1743737214103),
np.float64(1548.2834755459144)]
```

```
[82]: plt.plot(xaxis, hg_spectrum)
plt.plot(guessed_xvals, [100]*3, 'x')
plt.plot(improved_xval_guesses, [100]*3, '+');
```





Para nuestros valores de laboratorio tenemos :

```
[148]: improved_xval_guesseskr = [np.average(xvals5[g-npixels:g+npixels],
                                             weights=kru_spectrum[g-npixels:g+npixels] ->
                                             np.median(kru_spectrum))
                                  for g in guessed_xvalskr]
improved_xval_guesseskr
```

```
[148]: [np.float64(289.2471906134491),
        np.float64(355.44496014403074),
        np.float64(391.8638877399576),
        np.float64(561.5341289596395)]
```

```
[149]: improved_xval_guessesne = [np.average(xvals6[g-npixels:g+npixels],
                                             weights=neu_spectrum[g-npixels:g+npixels] ->
                                             np.median(neu_spectrum))
                                  for g in guessed_xvalsne]
improved_xval_guessesne
```

```
[149]: [np.float64(178.36840792411726),
        np.float64(194.12985654267652),
        np.float64(253.9363525171177),
```

```

np.float64(296.74615389835867),
np.float64(352.38057477348787),
np.float64(390.28525655867435),
np.float64(427.7350186872895),
np.float64(493.62956367611457),
np.float64(527.9300033218012),
np.float64(587.2089225147835)]

```

```

[85]: npixels = 15
improved_xval_guesseshg = [np.average(xaxishg[g-npixels:g+npixels],
                                     weights=hgu_spectrum[g-npixels:g+npixels] -
                                     np.median(hgu_spectrum))
                           for g in guessed_xvalshg]
improved_xval_guesseshg

```

```

[85]: [np.float64(97.761922442058),
       np.float64(203.2097312397874),
       np.float64(306.2503437594759),
       np.float64(354.608465694678),
       np.float64(583.25020087513)]

```

Solo tenemos tres puntos de datos (tutorial), pero eso es suficiente para ajustar un modelo lineal y aún tenemos un punto libre para verificar que lo hicimos cerca de lo correcto:

Usamos un modelo `Linear1D` porque queremos usar su inverso más adelante (otros modelos no son invertibles)

```

[150]: wlmodelhg = Linear1D()
linfit_wlmodelhg = linfitter(model=wlmodelhg, x=improved_xval_guesses,
                             y=guessed_wavelengths)
wavelengths = linfit_wlmodelhg(xaxis) * u.nm
linfit_wlmodelhg

wlmodelkr = Linear1D()
linfit_wlmodelkr = linfitter(model=wlmodelkr, x=improved_xval_guesseskr,
                              y=guessed_wavelengthskr)
wavelengthskr = linfit_wlmodelkr(xvals5) * u.nm
linfit_wlmodelkr

wlmodelne = Linear1D()
linfit_wlmodelne = linfitter(model=wlmodelne, x=improved_xval_guessesne,
                              y=guessed_wavelengthsne)
wavelengthsne = linfit_wlmodelkr(xvals6) * u.nm
linfit_wlmodelne

wlmodelhg = Linear1D()
linfit_wlmodelhg = linfitter(model=wlmodelhg, x=improved_xval_guesseshg,
                              y=guessed_wavelengthshg)

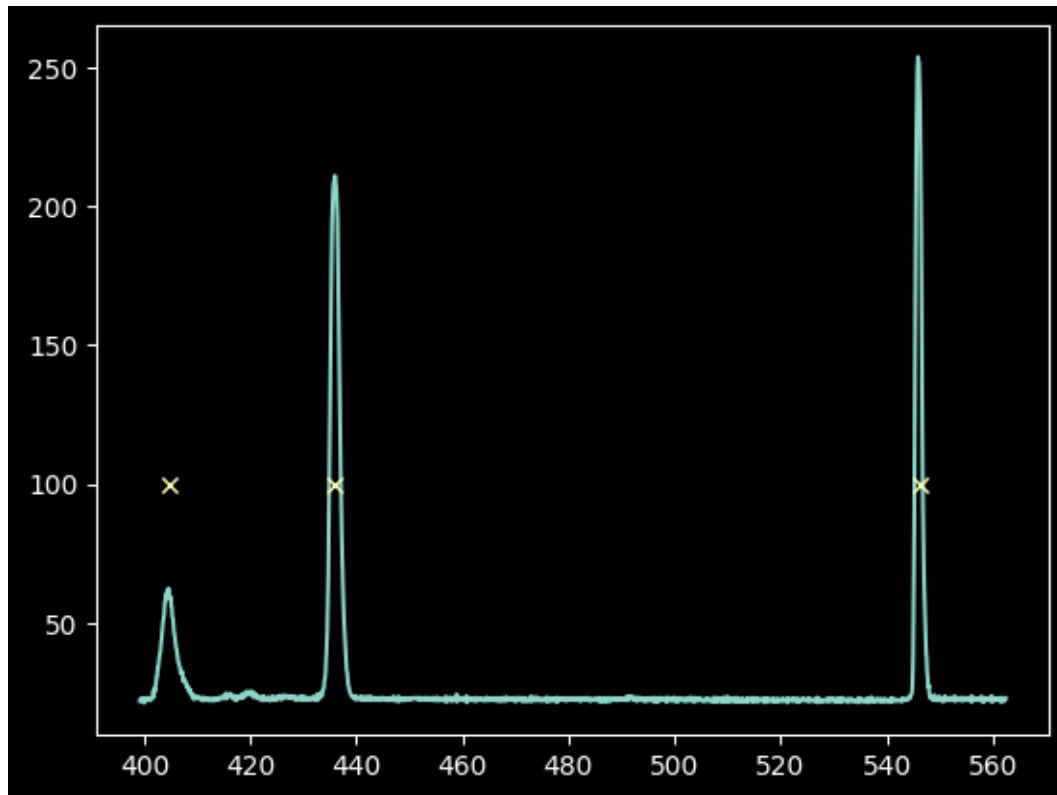
```

```
wavelengthshg = linfit_wlmodelhg(xaxisHg) * u.nm
linfit_wlmodelhg
```

[150]: <Linear1D(slope=0.53712037, intercept=356.98114063)>

Tenga en cuenta esta pendiente ajustada: cada píxel mide aproximadamente 0,1 nm (aproximadamente 1 angstrom) y la longitud de onda aumenta hacia la izquierda.

```
[151]: plt.plot(wavelengths, hg_spectrum)
plt.plot(guessed_wavelengths, [100]*3, 'x');
```



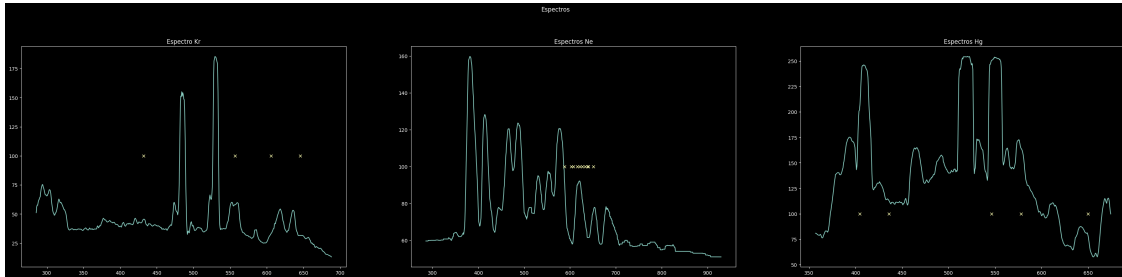
```
[153]: fig, (s, o, a) = plt.subplots(1, 3, figsize=(40, 8))
s.plot(wavelengthskr, kru_spectrum)
s.plot(guessed_wavelengthskr, [100]*4, 'x')
s.set_title("Espectro Kr")

o.plot(wavelengthsne, neu_spectrum)
o.plot(guessed_wavelengthsne, [100]*10, 'x')
o.set_title("Espectros Ne")

a.plot(wavelengthshg, hgu_spectrum)
a.plot(guessed_wavelengthshg, [100]*5, 'x')
```

```
a.set_title("Espectros Hg")
fig.suptitle("Espectros", y=1.02)
```

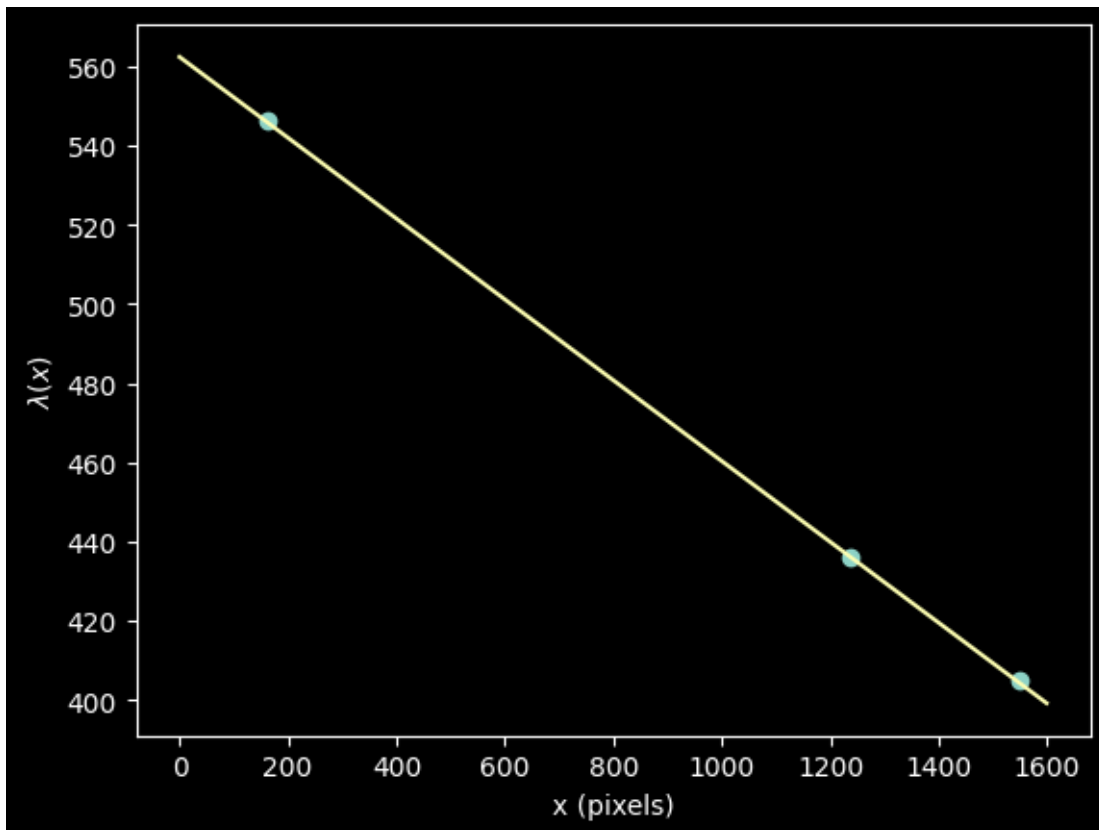
```
[153]: Text(0.5, 1.02, 'Espectros')
```



Mostramos nuestro modelo  $\lambda(x)$  frente a las “conjeturas” de entrada:

```
[99]: plt.plot(improved_xval_guesses, guessed_wavelengths, 'o')
plt.plot(xaxis, wavelengths, '-')
plt.ylabel("$\lambda(x)$")
plt.xlabel("x (pixels)")
```

```
[99]: Text(0.5, 0, 'x (pixels)')
```



Para nuestros espectros de laboratorio

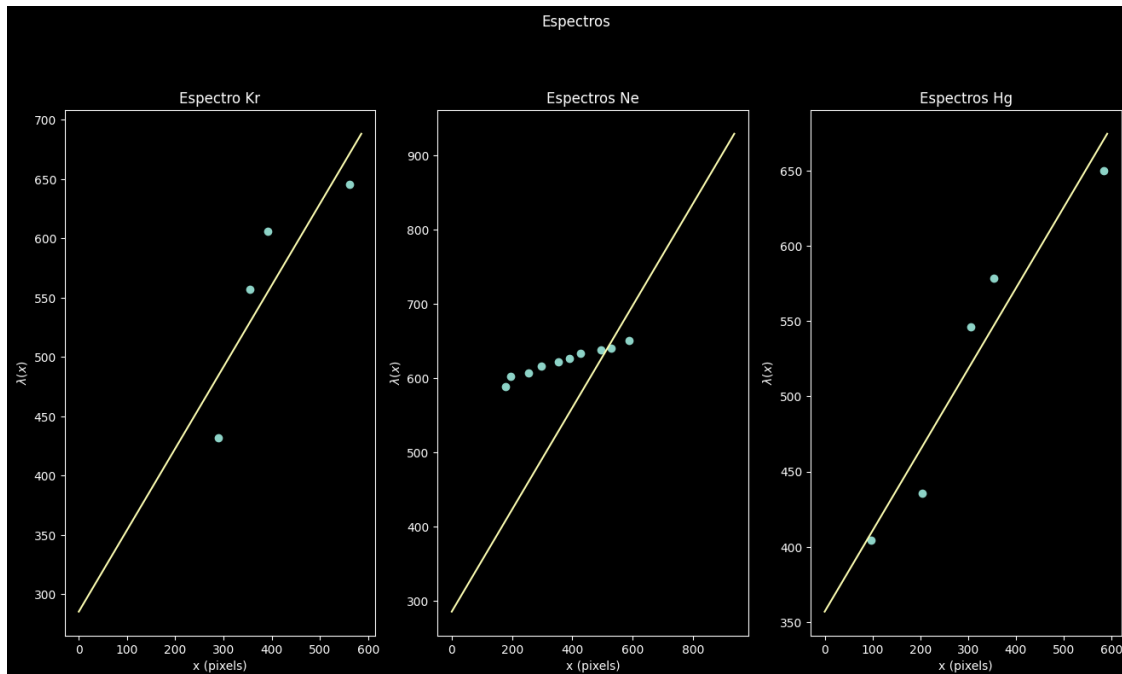
```
[154]: fig, (s, o ,a ) = plt.subplots(1,3, figsize=(16, 8))
s.plot(improved_xval_guesseskr, guessed_wavelengthskr, 'o')
s.plot(xvals5, wavelengthskr, '-')
s.set_ylabel("$\lambda(x)$") # + aquí estaba el error
s.set_xlabel("x (pixels)")
s.set_title("Espectro Kr")

o.plot(improved_xval_guessesne, guessed_wavelengthsne, 'o')
o.plot(xvals6, wavelengthsne, '-')
o.set_ylabel("$\lambda(x)$") # + aquí estaba el error
o.set_xlabel("x (pixels)")
o.set_title("Espectros Ne")

a.plot(improved_xval_guesseshg, guessed_wavelengthshg, 'o')
a.plot(xaxisshg, wavelengthshg, '-')
a.set_ylabel("$\lambda(x)$") # + aquí estaba el error
a.set_xlabel("x (pixels)")
a.set_title("Espectros Hg")

fig.suptitle("Espectros", y=1.02)
```

```
[154]: Text(0.5, 1.02, 'Espectros')
```



¡De hecho, un modelo lineal encaja perfectamente! No mucho para los espectros del lab

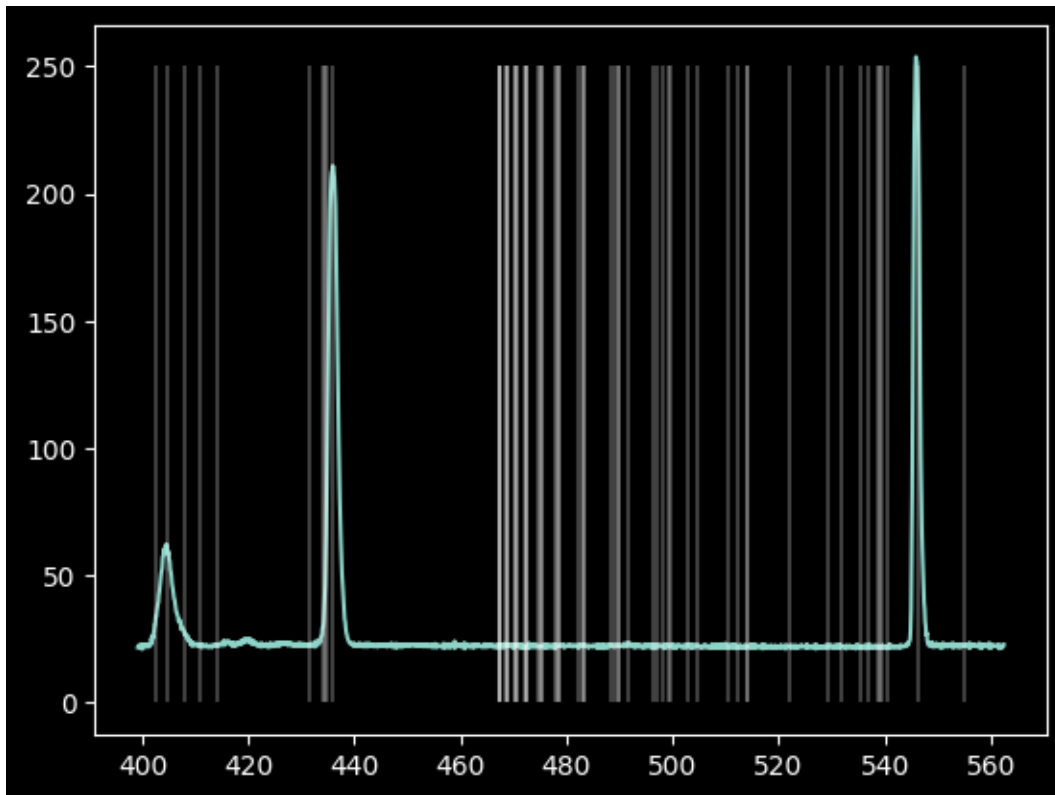
Podemos consultar las listas de líneas de neón y kriptón para ver qué tan bien lo hicimos y quizás intentar mejorar nuestro ajuste.

El NIST, el Instituto Nacional de Estándares y Tecnología, mantiene [listas de líneas atómicas](#). Sin embargo, no tenemos una forma directa de saber qué transiciones de estos átomos usar. Sin embargo, existen algunas heurísticas que podemos aplicar.

```
[155]: # we adopt the minimum/maximum wavelength from our linear fit
minwave = wavelengths.min()
maxwave = wavelengths.max()
# then we search for atomic lines
# We are only interested in neutral lines, assuming the lamps are not hot
    ↪ enough to ionize the atoms
mercury_lines = Nist.query(minwav=minwave,
                           maxwav=maxwave,
                           linename='Hg I')
krypton_lines = Nist.query(minwav=minwave,
                           maxwav=maxwave,
                           linename='Kr I')
neon_lines = Nist.query(minwav=minwave,
                        maxwav=maxwave,
                        linename='Ne I')
```

Primero, podemos observar lo que informa el NIST para el espectro de mercurio que ya ajustamos: (Tenga en cuenta que las longitudes de onda corresponden al *aire*, no al vacío).

```
[133]: plt.plot(wavelengths, hg_spectrum)
plt.vlines(mercury_lines['Observed'], 0, 250, 'w', alpha=0.25);
```

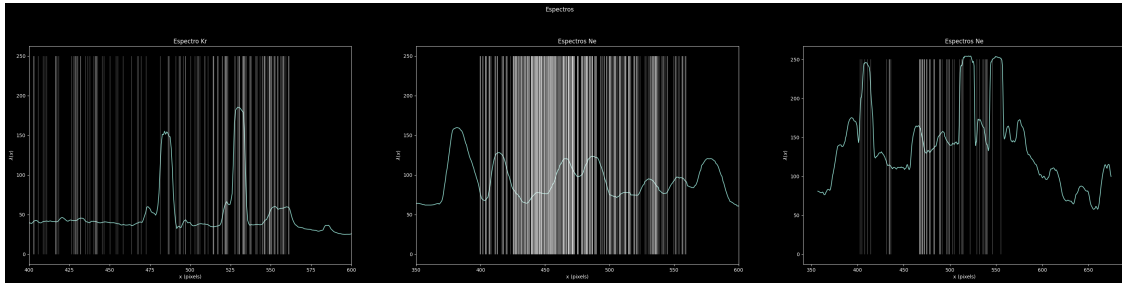


```
[161]: fig, (s, o ,a ) = plt.subplots(1,3 , figsize=(40, 8))
s.plot(wavelengthskr, kru_spectrum)
s.vlines(krypton_lines['Observed'], 0, 250, 'w', alpha=0.25);
s.set_xlim(400, 600)
s.set_ylabel("$\lambda(x)$")
s.set_xlabel("x (pixels)")
s.set_title("Espectro Kr")

o.plot(wavelengthsne, neu_spectrum)
o.vlines(neon_lines['Observed'], 0, 250, 'w', alpha=0.25);
o.set_xlim(350, 600)
o.set_ylabel("$\lambda(x)$")
o.set_xlabel("x (pixels)")
o.set_title("Espectros Ne")

a.plot(wavelengthshg, hgu_spectrum)
a.vlines(mercury_lines['Observed'], 0, 250, 'w', alpha=0.25);
a.set_ylabel("$\lambda(x)$")
a.set_xlabel("x (pixels)")
a.set_title("Espectros Ne")
fig.suptitle("Espectros", y=1.02)
```

```
[161]: Text(0.5, 1.02, 'Espectros')
```



La base de datos contiene *muchas* más líneas de las que vimos.

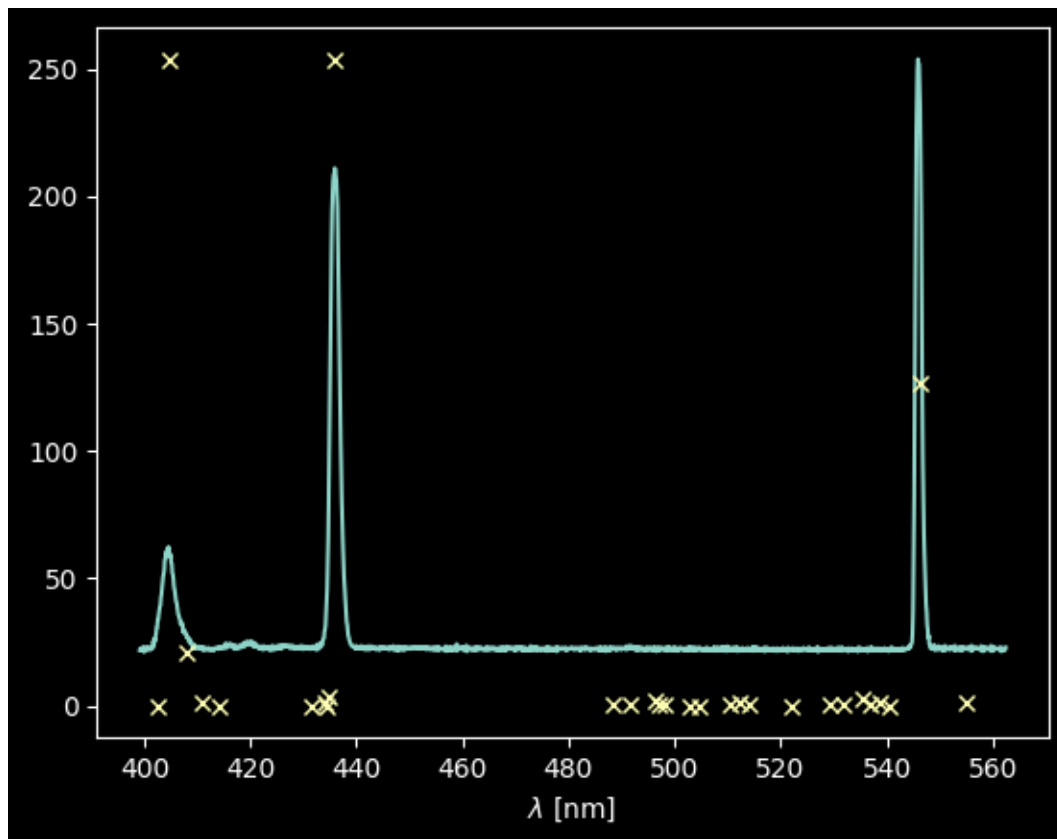
La “Intensidad Relativa” (Rel. Int) proporciona una descripción cualitativa del espectro de emisión de un elemento específico en una fuente específica (de baja densidad).

Podemos seleccionar esa columna si la depuramos un poco (hay algunas entradas en blanco).

```
[ ]: # these lines downselect from the table to keep only those that have usable
    ↪ "Relative Intensity" measurements
# first, we get rid of those whose 'Rel.' column is masked out or is an asterisk
hg_keep = (~mercury_lines['Rel.'].mask) & (mercury_lines['Rel.']. != "*")
hg_wl_tbl = mercury_lines['Observed'][hg_keep]
# then, we collect the 'Rel.' values and convert them from strings to floats
hg_rel_tbl = np.array([float(x) for x in mercury_lines['Rel.'][hg_keep]])

[ ]: plt.plot(wavelengths, hg_spectrum)
# we normalize the relative intensities to match the intensity of the spectrum
    ↪ so we can see both on the same plot
# since they're just relative intensities, their amplitudes are arbitrary anyway
plt.plot(hg_wl_tbl, hg_rel_tbl / hg_rel_tbl.max() * hg_spectrum.max(), 'x')
plt.xlabel("$\lambda$ [nm]");
```





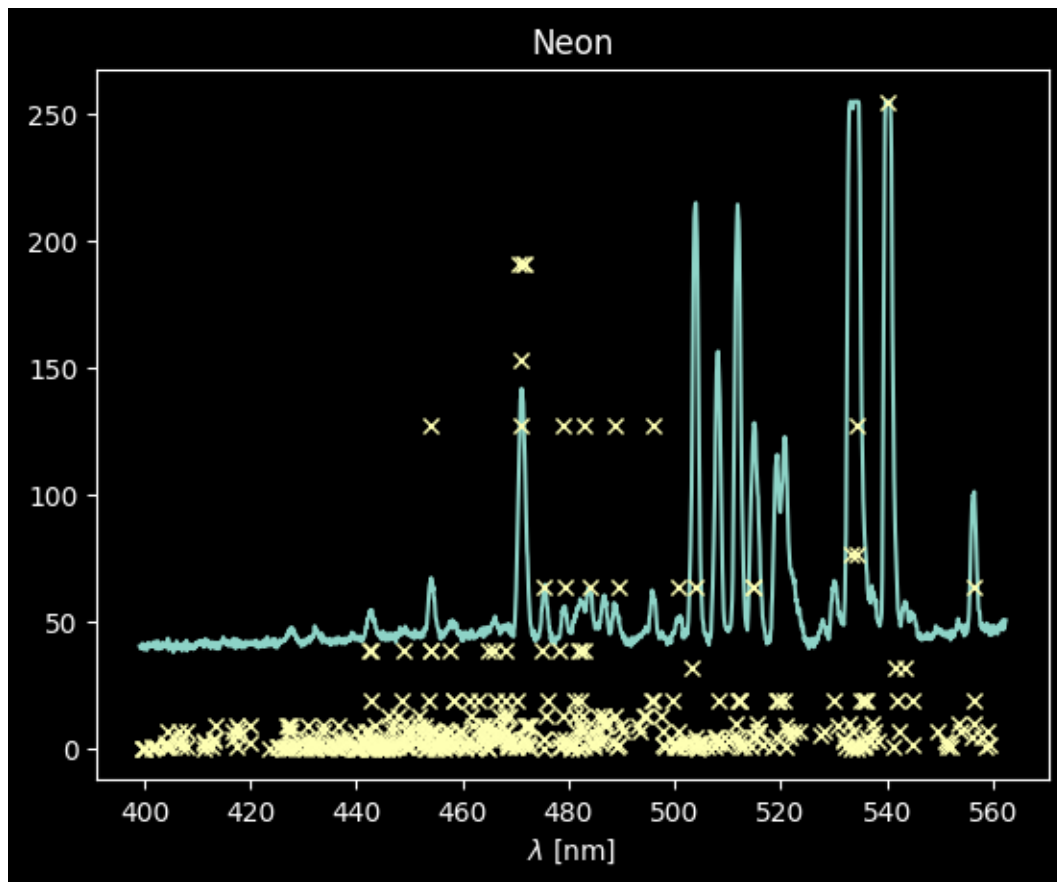
¡Bien! Fíjate que solo se ven las tres líneas más brillantes, y quizás una tercera alrededor de los 408 nm.

Intentemos lo mismo con las demás líneas (ten en cuenta que la técnica cambia; esto se debe a que las tablas del NIST no están formateadas para ser legibles por máquina).

```
[ ]: ne_keep = np.array(['*' not in x for x in neon_lines['Rel.']]
ne_wl_tbl = neon_lines['Observed'][ne_keep]
ne_rel_tbl = np.array([float(x) for x in neon_lines['Rel.'][ne_keep]])

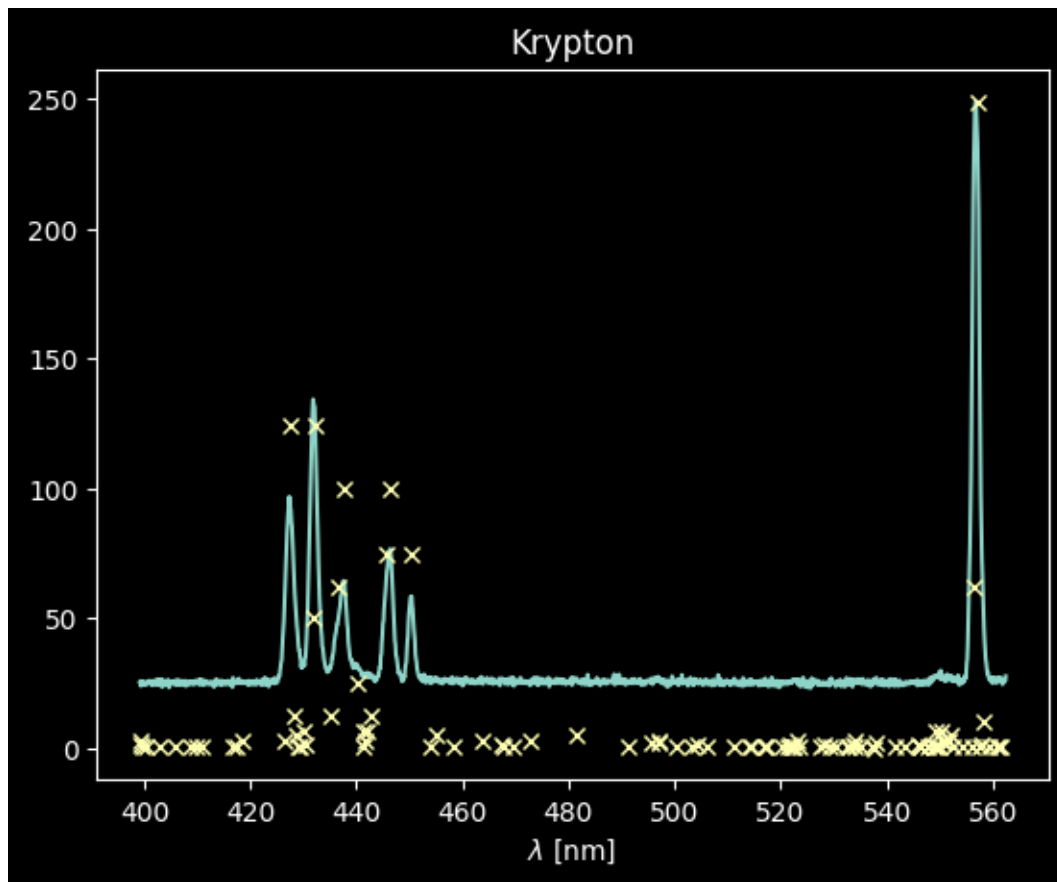
[ ]: kr_keep = np.array(['*' not in x) and ('h' not in x) and ('-' not in x) for x_
    ↪in krypton_lines['Rel.']]
kr_wl_tbl = krypton_lines['Observed'][kr_keep]
kr_rel_tbl = np.array([float(x) for x in krypton_lines['Rel.'][kr_keep]])

[ ]: plt.plot(wavelengths, ne_spectrum)
plt.plot(ne_wl_tbl, ne_rel_tbl / ne_rel_tbl.max() * ne_spectrum.max(), 'x')
plt.xlabel("$\lambda$ [nm]")
plt.title("Neon");
```



En el caso de Neon, hay muchas líneas que coinciden, pero muchas que no coinciden bien (detectamos muchas que tienen valores “Rel” bajos)

```
[ ]: plt.plot(wavelengths, kr_spectrum)
plt.plot(kr_wl_tbl, kr_rel_tbl / kr_rel_tbl.max() * kr_spectrum.max(), 'x')
plt.xlabel("$\lambda$ [nm]")
plt.title("Krypton");
```



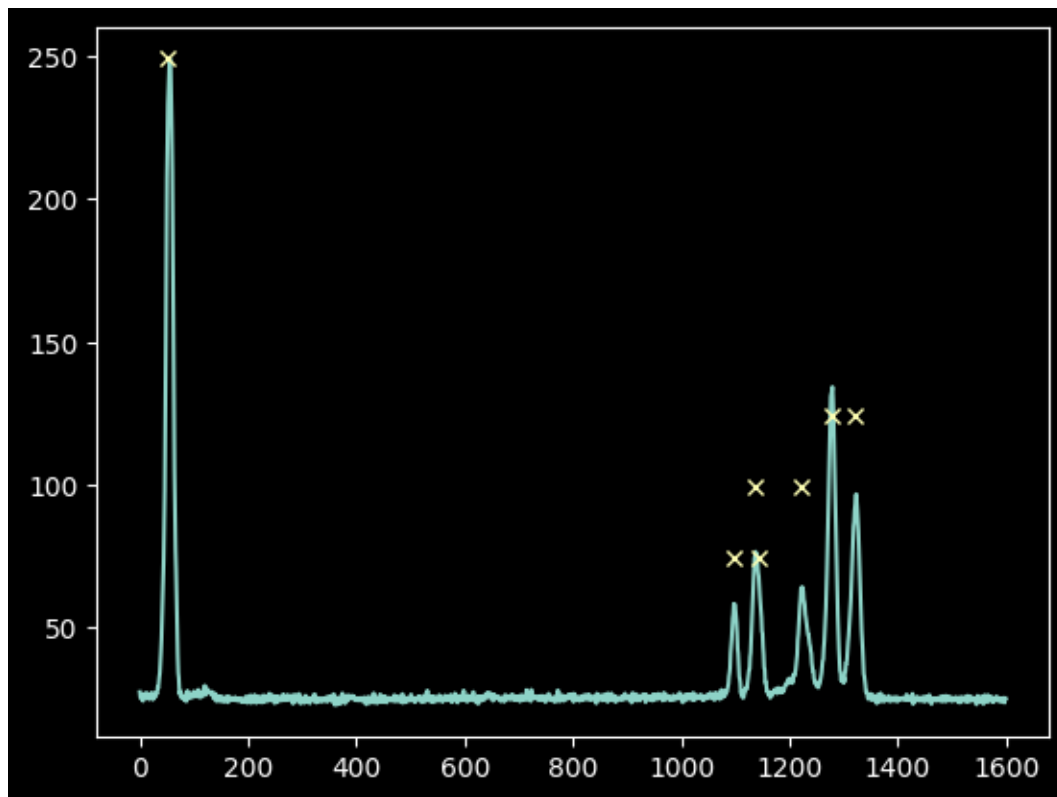
El espectro de criptón será mucho más fácil de manejar: simplemente seleccionamos aquellas líneas con intensidades  $> 70$  en la escala anterior:

```
[ ]: kr_rel_intens = kr_rel_tbl / kr_rel_tbl.max() * kr_spectrum.max()
kr_keep_final = kr_rel_intens > 70
kr_wl_final = kr_wl_tbl[kr_keep_final]
```

Luego tomamos estas longitudes de onda de criptón y las convertimos nuevamente al espacio de píxeles:

```
[ ]: # the linear model has an inverse function that takes  $y = m x + b$  and converts
      ↪ to  $x = (y - b) / m$ 
kr_pixel_vals = linfit_wlmodel.inverse(kr_wl_final)

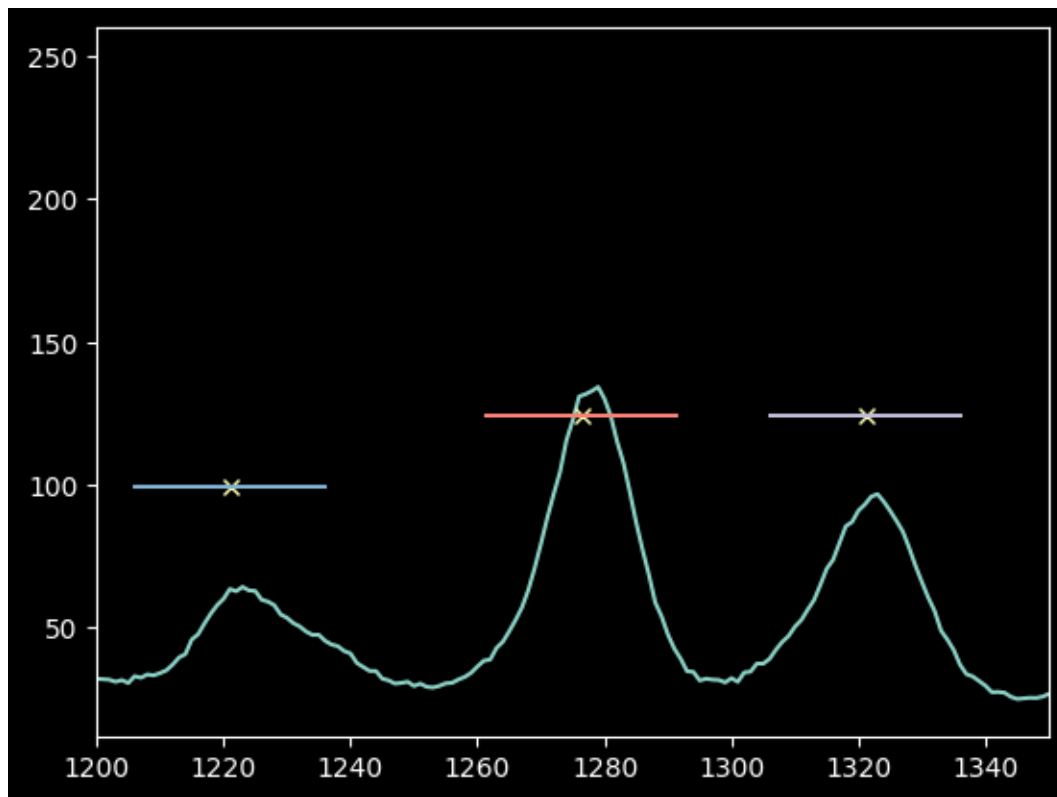
[ ]: plt.plot(xaxis, kr_spectrum)
plt.plot(kr_pixel_vals, kr_rel_intens[kr_keep_final], 'x');
```



Al igual que con el mercurio mencionado anteriormente, seleccionamos una subregión y medimos la posición del pico ponderado por intensidad en número de píxeles para medir con precisión el centro del píxel.

¡Pero aquí debemos tener cuidado! ¡Estos picos están muy cerca! ¿Podemos seguir usando un rango de píxeles de  $\pm 15$ ?

```
[ ]: # Examine the pixel range, +/-15 pix, to see if it results in line overlaps
plt.plot(xaxis, kr_spectrum)
plt.plot(kr_pixel_vals, kr_rel_intens[kr_keep_final], 'x')
for xx, yy in zip(kr_pixel_vals, kr_rel_intens[kr_keep_final]):
    plt.plot([xx-15,xx+15], [yy,yy], ) # plot horizontal lines at each peak
    ↪location
plt.xlim(1200,1350);
```



Sí, 15 píxeles todavía se ven bien.

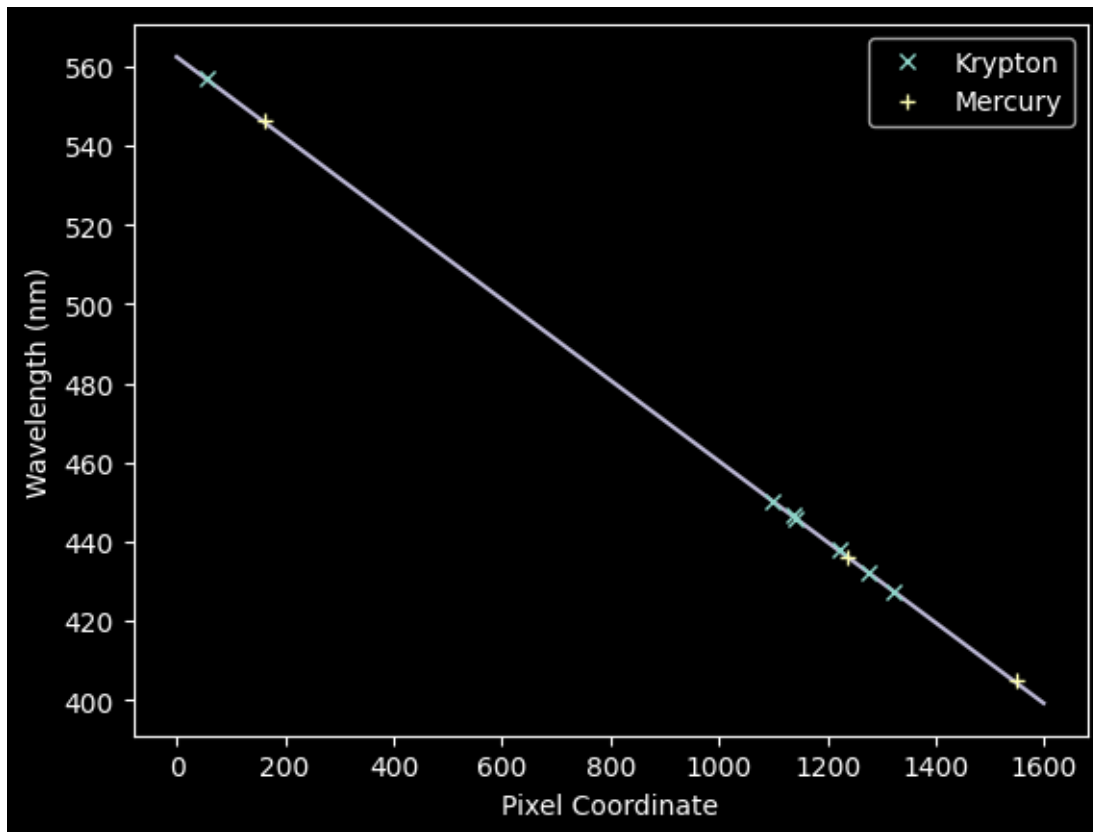
Si estas líneas se hubieran superpuesto, habríamos querido reducir el rango de píxeles.

```
[ ]: # note that we have to force the pixel-center guesses to be integers for
      ↪slicing:
      npixels = 15
      improved_xval_guesses_kr = [np.average(xaxis[g-npixels:g+npixels],
                                             weights=kr_spectrum[g-npixels:g+npixels]
      ↪np.median(kr_spectrum))
                                for g in map(int, kr_pixel_vals)]
      improved_xval_guesses_kr
```

```
[ ]: [np.float64(1321.3688120277354),
      np.float64(1277.1972221744618),
      np.float64(1222.9735220981242),
      np.float64(1140.2780334574877),
      np.float64(1137.7518421055224),
      np.float64(1097.715582422276),
      np.float64(54.2769391627054)]
```

Ahora podemos graficar estos números de píxeles de mejor ajuste contra las longitudes de onda “reales” y superponer nuestro modelo de mejor ajuste anterior de mercurio:

```
[ ]: plt.plot(improved_xval_guesses_kr, kr_wl_final, 'x', label='Krypton')
plt.plot(improved_xval_guesses, guessed_wavelengths, '+', label='Mercury')
plt.plot(xaxis, wavelengths, zorder=-5)
plt.legend(loc='best')
plt.xlabel("Pixel Coordinate")
plt.ylabel("Wavelength (nm)");
```



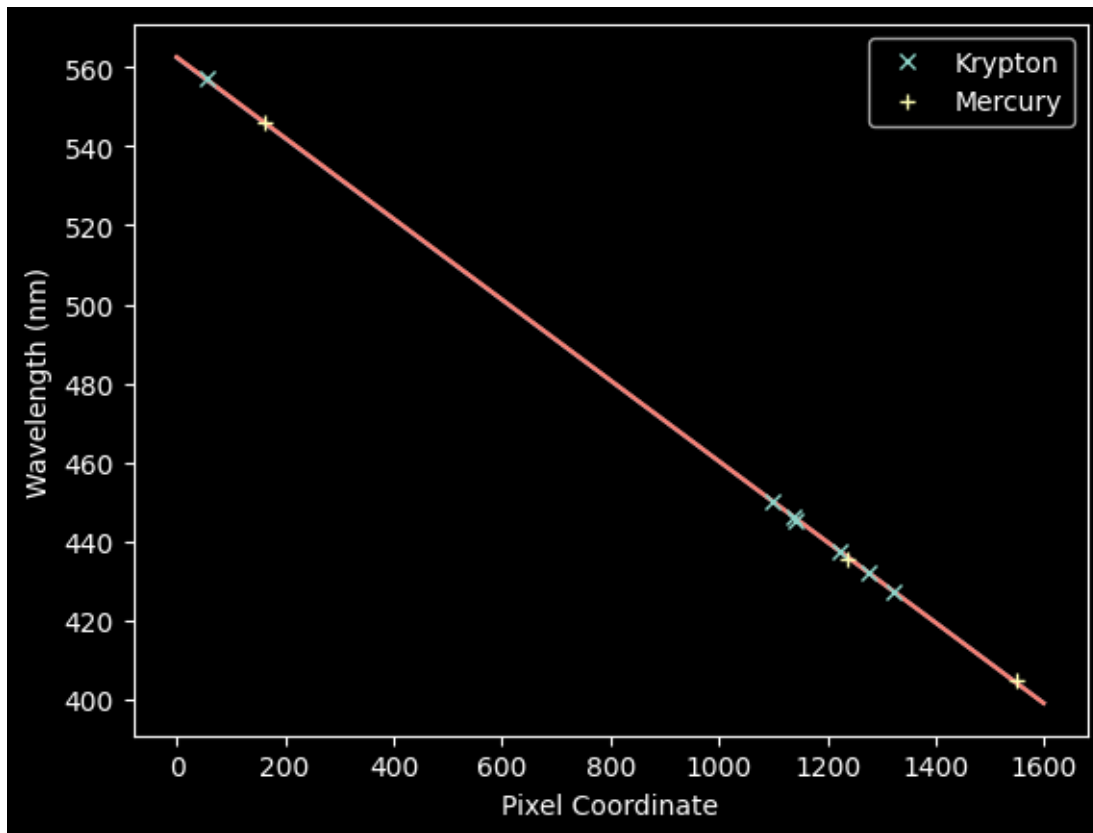
Ahora podemos ajustar conjuntamente los espectros de Kr + Hg con un nuevo modelo:

```
[ ]: # we concatenate the data sets together
xvals_hg_plus_kr = np.concatenate([improved_xval_guesses,
                                     improved_xval_guesses_kr])
waves_hg_plus_kr = np.concatenate([guessed_wavelengths, kr_wl_final])
linfit_wlmodel_hgkr = linfitter(model=wlmodel,
                                x=xvals_hg_plus_kr,
                                y=waves_hg_plus_kr)

linfit_wlmodel_hgkr
```

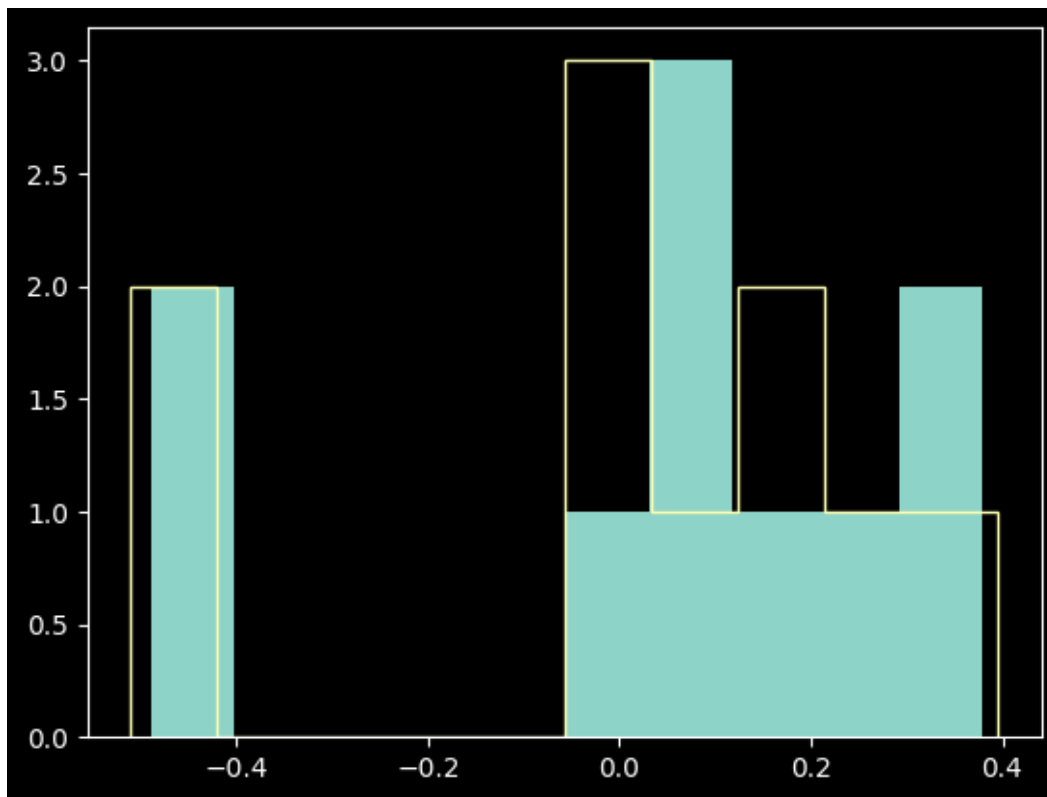
```
[ ]: <Linear1D(slope=-0.10221918, intercept=562.56900643)>
```

```
[ ]: # re-plot the data with the fit overlaid
plt.plot(improved_xval_guesses_kr, kr_wl_final, 'x', label='Krypton')
plt.plot(improved_xval_guesses, guessed_wavelengths, '+', label='Mercury')
plt.plot(xaxis, wavelengths, zorder=-5)
plt.plot(xaxis, linfit_wlmodel_hgkr(xaxis), zorder=-5)
plt.legend(loc='best')
plt.xlabel("Pixel Coordinate")
plt.ylabel("Wavelength (nm)");
```



Son casi indistinguibles, pero veamos los residuos de cada uno:

```
[ ]: residuals_hgonlymodel = np.array(waves_hg_plus_kr) -
    ↳linfit_wlmodel(xvals_hg_plus_kr)
residuals_hgkrmodel = np.array(waves_hg_plus_kr) -
    ↳linfit_wlmodel_hgkr(xvals_hg_plus_kr)
plt.hist(residuals_hgonlymodel, label='Hg-only fit', histtype='stepfilled')
plt.hist(residuals_hgkrmodel, label='Hg-only fit', histtype='step');
```

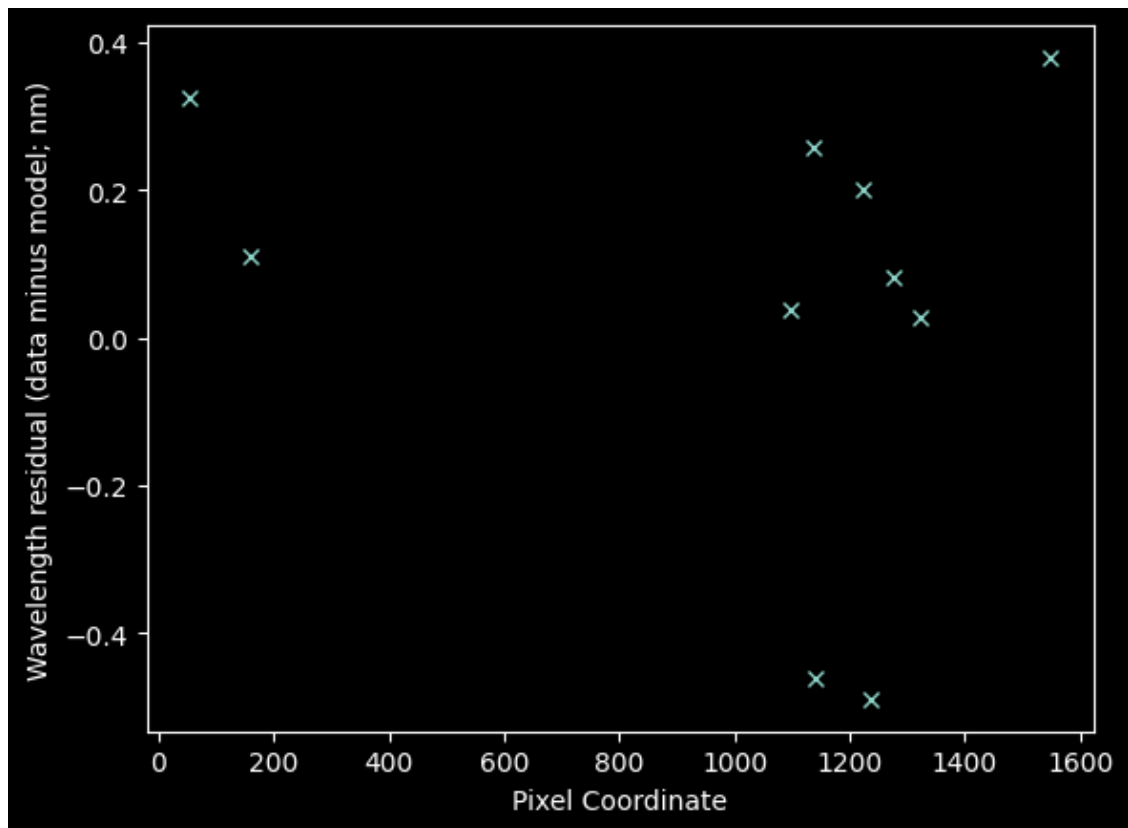


Las diferencias son bastante marginales; el ajuste solo con Hg fue realmente bueno.

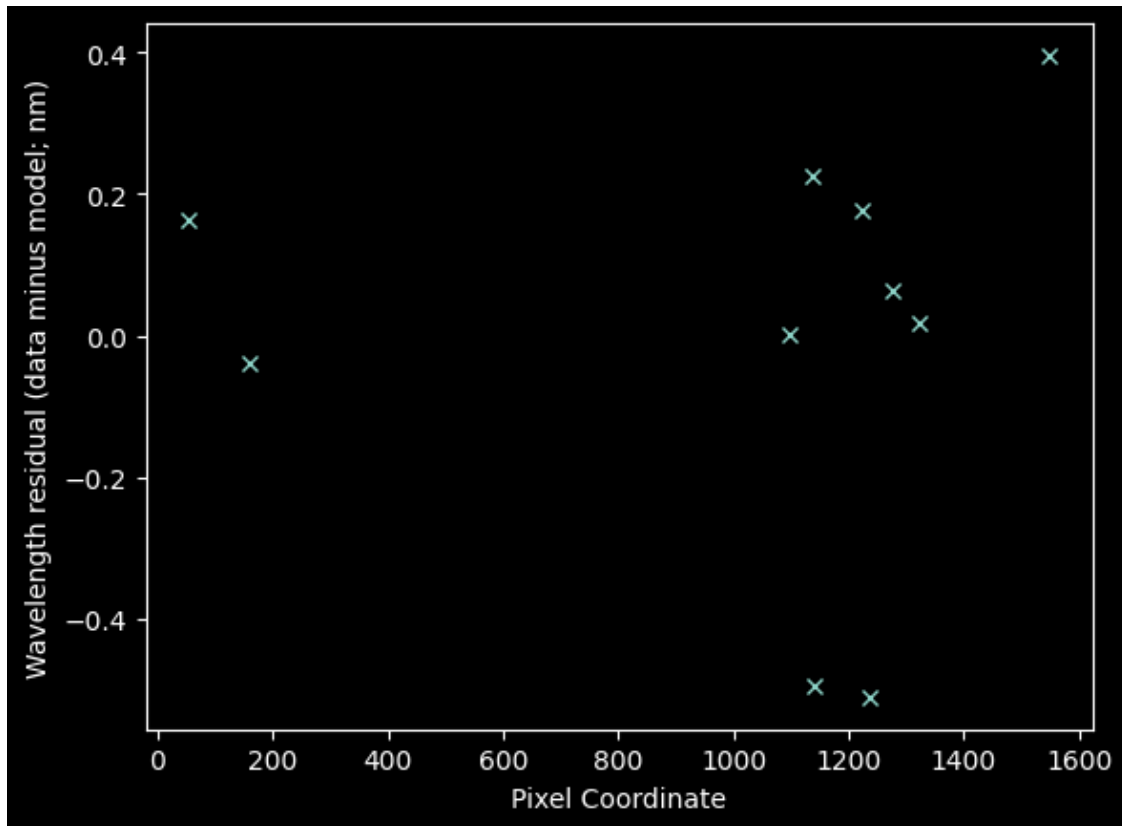
¿Tienen alguna forma los residuos?

```
[ ]: plt.plot(xvals_hg_plus_kr, residuals_hgonlymodel, 'x')
plt.xlabel("Pixel Coordinate")
plt.ylabel("Wavelength residual (data minus model; nm)");
```





```
[ ]: plt.plot(xvals_hg_plus_kr, residuals_hgkrmodel, 'x')
plt.xlabel("Pixel Coordinate")
plt.ylabel("Wavelength residual (data minus model; nm)");
```



En resumen, no, no hay indicios de estructura. Si la hubiera, quizá convendría ajustar un modelo de orden superior.

También podemos estimar una estadística similar a chi-cuadrado (aunque tenga en cuenta que este *no* es el valor  $\chi^2$  que usaríamos para evaluar la bondad del ajuste porque *no* hemos estimado la incertidumbre en cada punto de longitud de onda).

```
[ ]: resid2_hgonly = (residuals_hgonlymodel**2).sum()
      resid2_hgkr = (residuals_hgkrmodel**2).sum()
      resid2_hgonly, resid2_hgkr
```

```
[ ]: (np.float64(0.8290377437520076), np.float64(0.7755414146718549))
```

Estas diferencias también son muy pequeñas, lo que nuevamente sugiere que el ajuste solo de Hg fue realmente bueno.

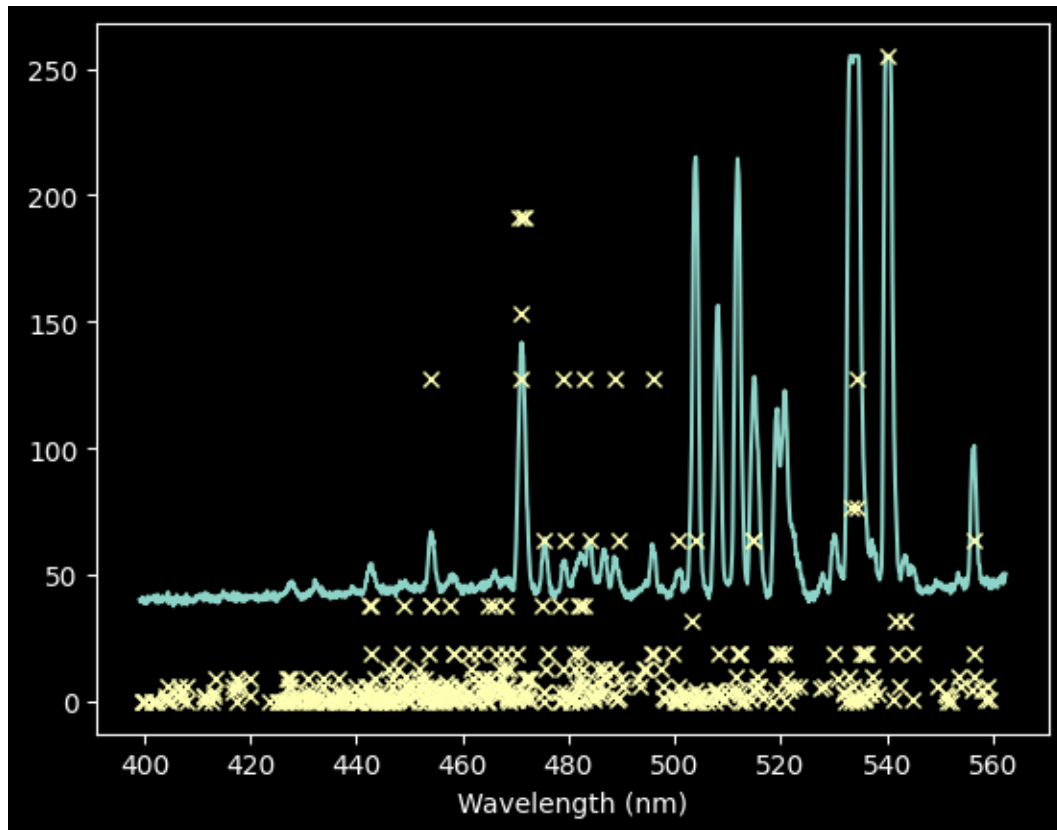
¿Por qué nos molestamos en realizar todas estas comprobaciones si el ajuste solo de Hg fue perfecto?

Además de simplemente desconocerlo, es común que los espectrógrafos presenten una dispersión ligeramente no lineal al medirse en un amplio rango de longitudes de onda, por lo que queremos asegurarnos de no haber pasado por alto ninguna de estas curvaturas.

También queremos medir la incertidumbre en nuestra calibración de longitud de onda.

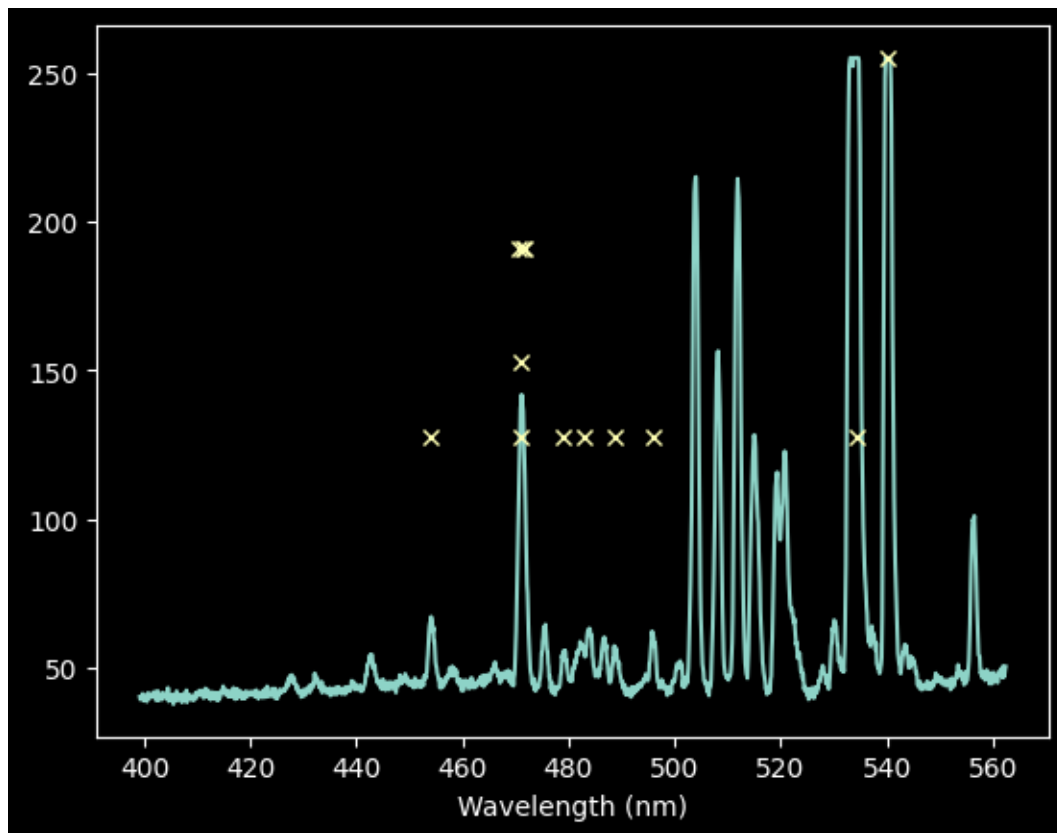
Ahora volvamos y midamos el espectro de neón.

```
[ ]: ne_rel_intens = ne_rel_tbl / ne_rel_tbl.max() * ne_spectrum.max()
plt.plot(wavelengths, ne_spectrum)
plt.plot(ne_wl_tbl, ne_rel_intens, 'x')
plt.xlabel('Wavelength (nm)');
```



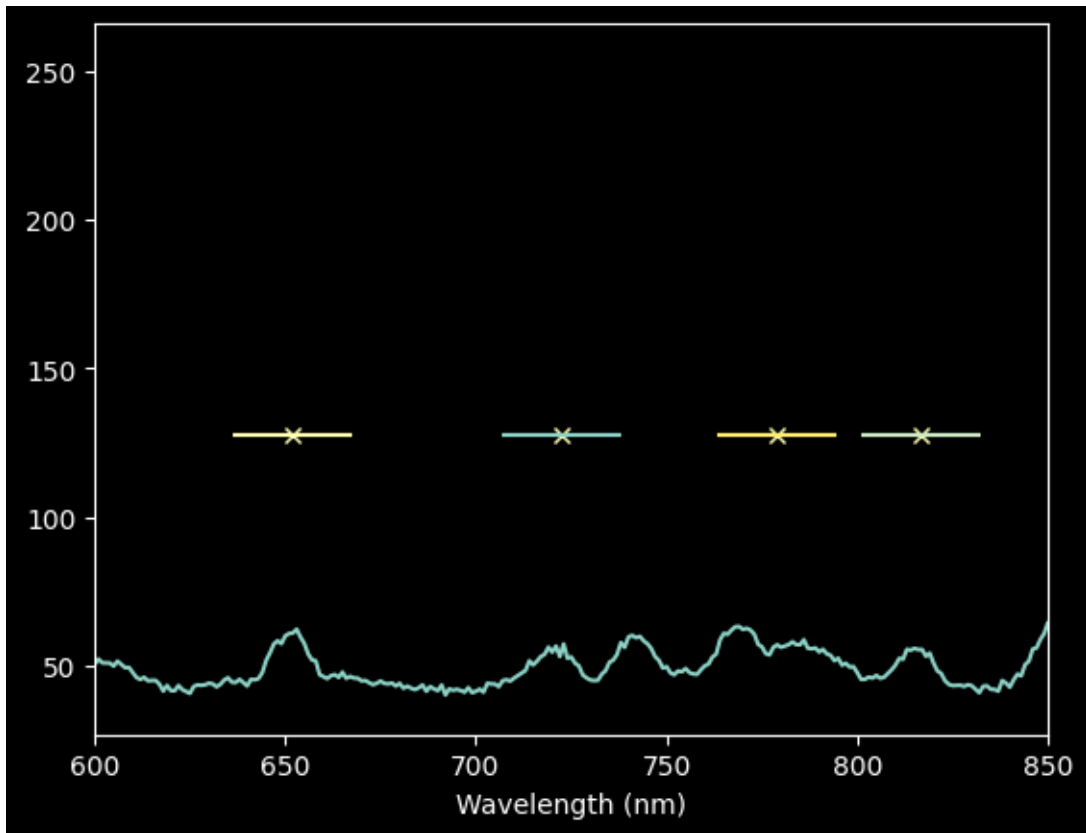
Podemos intentar mantener sólo aquellos con intensidad >100, aunque perderemos algunas líneas.

```
[ ]: ne_keep_final = ne_rel_intens > 100
plt.plot(wavelengths, ne_spectrum)
plt.plot(ne_wl_tbl[ne_keep_final], ne_rel_intens[ne_keep_final], 'x')
plt.xlabel('Wavelength (nm)');
```



```
[ ]: # select down to just those lines we want to keep
ne_wl_final = ne_wl_tbl[ne_keep_final]
ne_pixel_vals = linfit_wlmodel.inverse(ne_wl_final)

[ ]: # check for overlaps again (in the densest part of the spectrum)
plt.plot(xaxis, ne_spectrum)
plt.plot(ne_pixel_vals, ne_rel_intens[ne_keep_final], 'x')
for xx, yy in zip(ne_pixel_vals, ne_rel_intens[ne_keep_final]):
    plt.plot([xx-15,xx+15], [yy,yy], )
plt.xlim(600,850)
plt.xlabel('Wavelength (nm)');
```



Para Neón, nuestro enfoque automatizado no es muy bueno (el tercer punto desde la izquierda no está en un pico), y preferimos usar un rango de píxeles más estrecho.

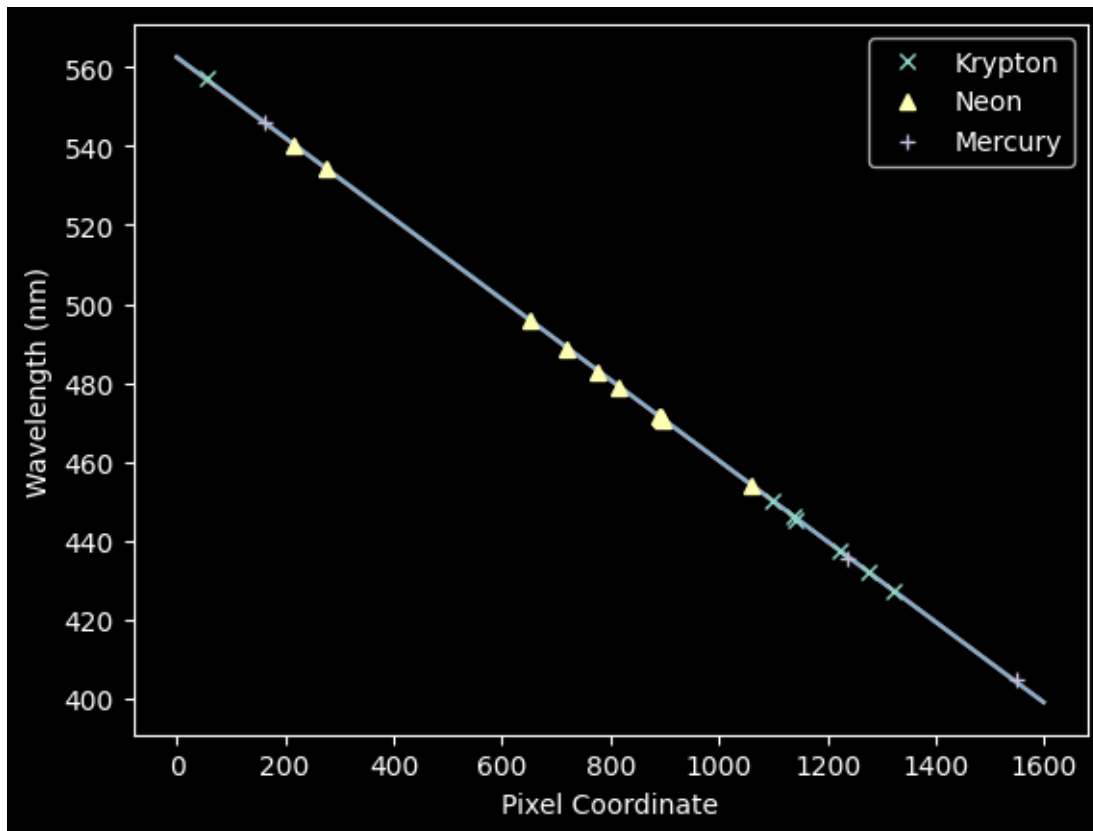
Aun así, en lugar de retroceder e identificar laboriosamente cada línea, nos quedaremos con un enfoque semiautomatizado, usando simplemente un rango más estrecho alrededor de cada pico.

```
[ ]: npixels = 10
improved_xval_guesses_ne = [np.average(xaxis[g-npixels:g+npixels],
                                     weights=ne_spectrum[g-npixels:g+npixels] -
                                     np.median(ne_spectrum))
                             for g in map(int, ne_pixel_vals)]
improved_xval_guesses_ne
```

```
[ ]: [np.float64(1060.5183193475166),
      np.float64(895.2424884504669),
      np.float64(893.670848263007),
      np.float64(892.9232835842486),
      np.float64(892.082009702997),
      np.float64(890.6754750779604),
      np.float64(814.5085554585149),
      np.float64(776.6287593955387),
      np.float64(720.1158679501369),
```

```
np.float64(651.4261979644568),
np.float64(275.48231610553455),
np.float64(216.88320524067413)]
```

```
[ ]: plt.plot(improved_xval_guesses_kr, kr_wl_final, 'x', label='Krypton')
plt.plot(improved_xval_guesses_ne, ne_wl_final, '^', label='Neon')
plt.plot(improved_xval_guesses, guessed_wavelengths, '+', label='Mercury')
plt.plot(xaxis, wavelengths, zorder=-5)
plt.plot(xaxis, linfit_wlmodel_hgkr(xaxis), zorder=-5)
plt.legend(loc='best')
plt.xlabel("Pixel Coordinate")
plt.ylabel("Wavelength (nm)");
```



Concatenamos las líneas de Neón en el espectro y volvemos a ajustarlas.

```
[ ]: xvals_hg_plus_kr_plus_ne = list(improved_xval_guesses) +
    ↳ list(improved_xval_guesses_kr) + list(improved_xval_guesses_ne)
waves_hg_plus_kr_plus_ne = list(guessed_wavelengths) + list(kr_wl_final) +
    ↳ list(ne_wl_final)
linfit_wlmodel_hgkrne = linfitter(model=wlmodel, x=xvals_hg_plus_kr_plus_ne,
    ↳ y=waves_hg_plus_kr_plus_ne)
```

```
linfit_wlmodel_hgkrne
```

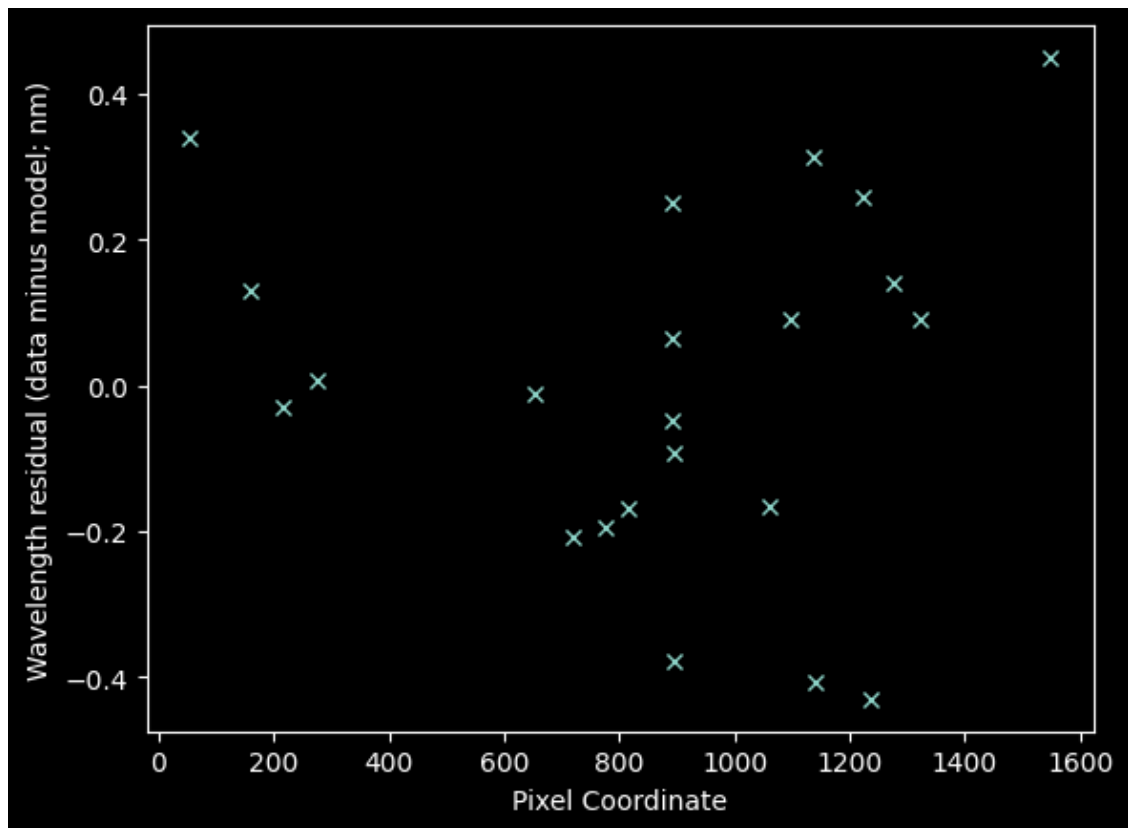
```
[ ]: <Linear1D(slope=-0.10213663, intercept=562.38704033)>
```

```
[ ]: residuals_hgmodel_nedata = np.array(waves_hg_plus_kr_plus_ne) -  
    ↪ linfit_wlmodel(xvals_hg_plus_kr_plus_ne)  
residuals_hgkrmodel_nedata = np.array(waves_hg_plus_kr_plus_ne) -  
    ↪ linfit_wlmodel_hgkr(xvals_hg_plus_kr_plus_ne)  
residuals_hgkrnemodel_nedata = np.array(waves_hg_plus_kr_plus_ne) -  
    ↪ linfit_wlmodel_hgkrne(xvals_hg_plus_kr_plus_ne)
```

```
[ ]: # calculate and compare the squared residuals  
# the model calculated for Hg-only works better than that for Krypton, but the  
    ↪ optimal linear model is calculated  
# from all data, with a squared residual of 1.26 nm  
resid2_hgonly = (residuals_hgmodel_nedata**2).sum()  
resid2_hgkr = (residuals_hgkrmodel_nedata**2).sum()  
resid2_hgkrne = (residuals_hgkrnemodel_nedata**2).sum()  
resid2_hgonly, resid2_hgkr, resid2_hgkrne
```

```
[ ]: (np.float64(1.2974074905657398),  
      np.float64(1.5361847762873513),  
      np.float64(1.2470762011963794))
```

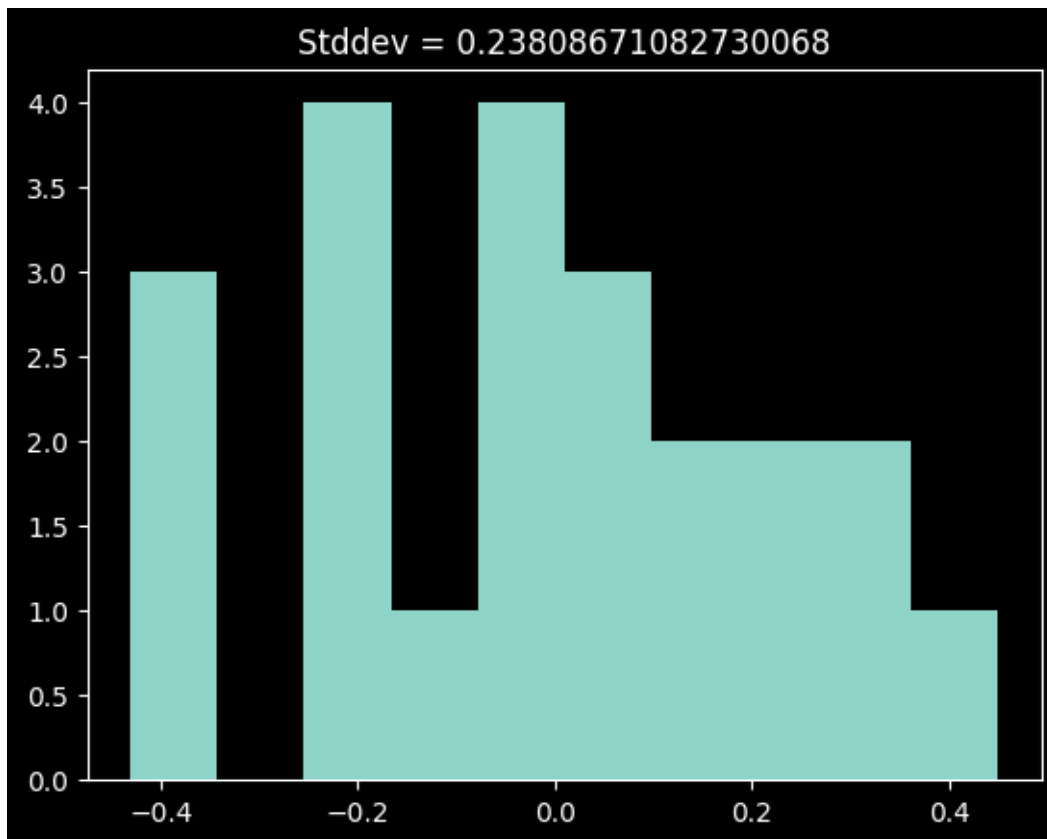
```
[ ]: plt.plot(xvals_hg_plus_kr_plus_ne, residuals_hgkrnemodel_nedata, 'x')  
plt.xlabel("Pixel Coordinate")  
plt.ylabel("Wavelength residual (data minus model; nm)");
```



Finalmente, con el neón incluido, hay un indicio de cierta estructura.

```
[ ]: plt.hist(residuals_hgkrnemodel_nedata)
plt.title(f"Stddev = {residuals_hgkrnemodel_nedata.std()}");
```





Este histograma quizás esté un poco sesgado.

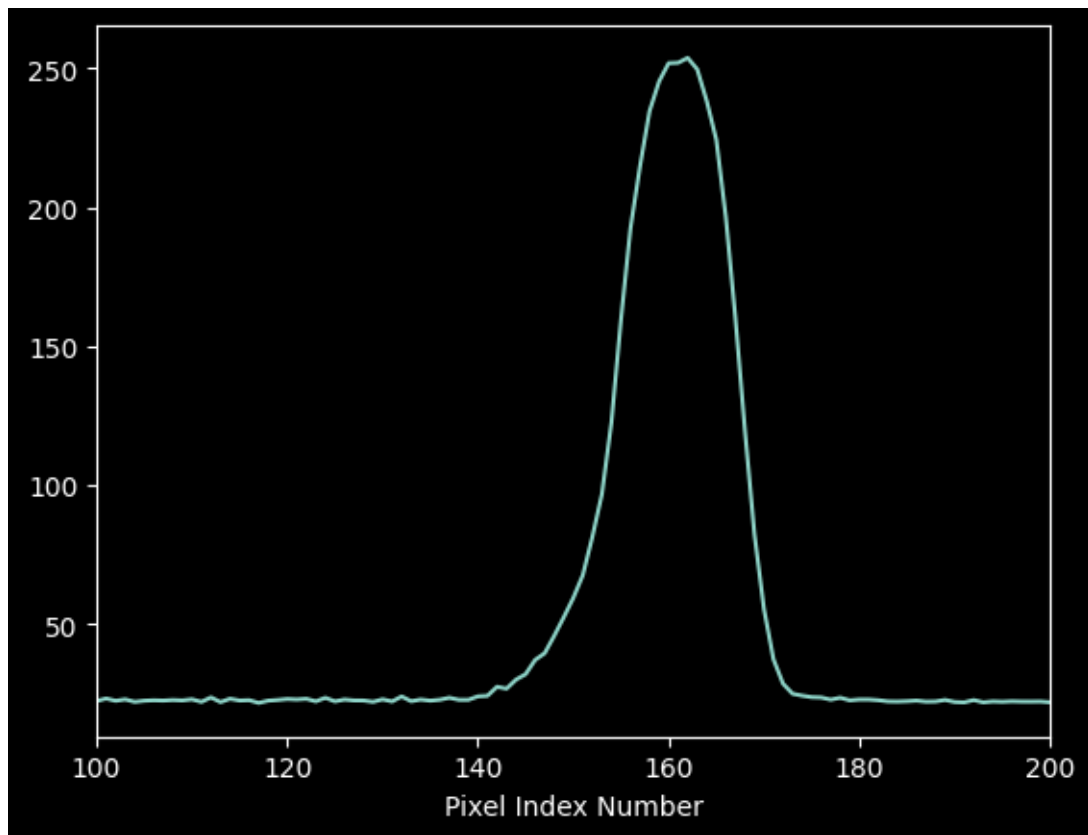
Puede haber cierta curvatura en este espectro, pero la evidencia no es muy convincente, en particular dada la naturaleza incierta de los ajustes de la línea Neon.

La dispersión de estos residuos es de aproximadamente 0,24 nm. Analizaremos este valor con más detalle más adelante.

#### 0.4 Medición de la incertidumbre de la solución de longitud de onda

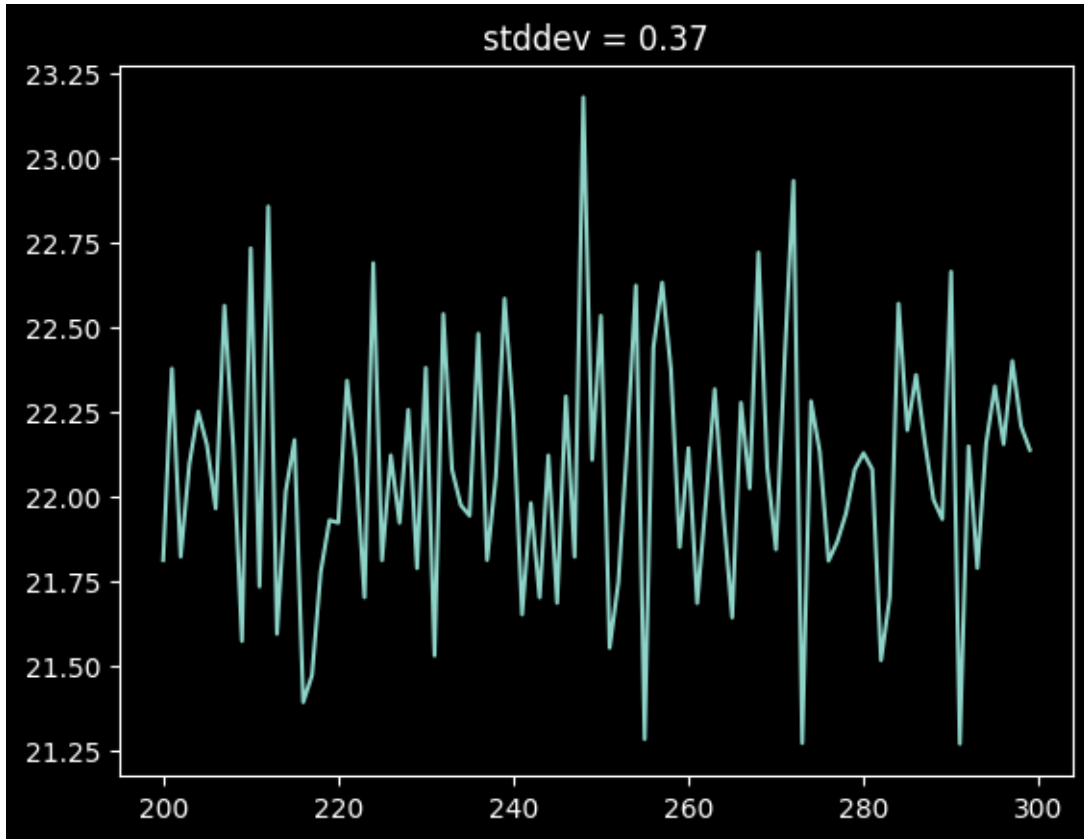
Para evaluar nuestra incertidumbre, primero analicemos la incertidumbre en el valor del píxel inferido al observar una sola línea en el espectro de mercurio.

```
[ ]: plt.plot(xaxis, hg_spectrum)
plt.xlim(100,200)
plt.xlabel("Pixel Index Number");
```



Podemos obtener una estimación empírica *aproximada* de la incertidumbre por píxel tomando la desviación estándar de una parte “en blanco” del espectro:

```
[ ]: plt.plot(xaxis[200:300], hg_spectrum[200:300])
      noise_estimate = hg_spectrum[200:300].std()
      plt.title(f"stddev = {noise_estimate:0.2f}");
```



The moment-1 estimate of the peak location is, for a spectrum  $f(x)$  and pixel location  $x$

$$m_1 = \frac{\sum x f(x)}{\sum f(x)}$$

which, through propagation of error, gives us variance of moment 1:

$$\sigma_{m_1}^2 = \left( \frac{\sum [(x - m_1)^2 \sigma_{f(x)}^2]}{(\sum f(x))^2} + \frac{\sigma_{\sum f(x)}^2 \sum [(x - m_1)^2 f(x)^2]}{(\sum f(x))^4} \right)$$

where  $\{f(x)\}^2 = \{f(x)\}^2 = N \{f(x)\}^2$

```
[ ]: cutout = hg_spectrum[100:200] - np.median(hg_spectrum)
      xcutout = xaxis[100:200]
      m1 = (xcutout * cutout).sum() / cutout.sum()
      m1
```

```
[ ]: np.float64(160.14071738720793)
```

```
[ ]: # Uncertainty on moment 1
      sigma_m1_left = ((xcutout-m1)**2 * noise_estimate**2).sum() / cutout.sum()**2
```

```
sigma_m1_right = (xcutout.size * noise_estimate**2) * \↪
↪((xcutout-m1)**2*cutout**2).sum() / cutout.sum()**4
sigma_m1 = sigma_m1_left + sigma_m1_right
sigma_m1
```

```
[ ]: np.float64(0.0012527166750909403)
```

```
[ ]: print(f"Moment analysis yields m1 = {m1:0.3f} +/- {sigma_m1:0.3f} pixels")
```

Moment analysis yields m1 = 160.141 +/- 0.001 pixels

La incertidumbre es de aproximadamente una milésima de píxel para una línea brillante. Puede ser algo mayor para líneas más tenues, pero sigue siendo mucho menor que la dispersión observada.

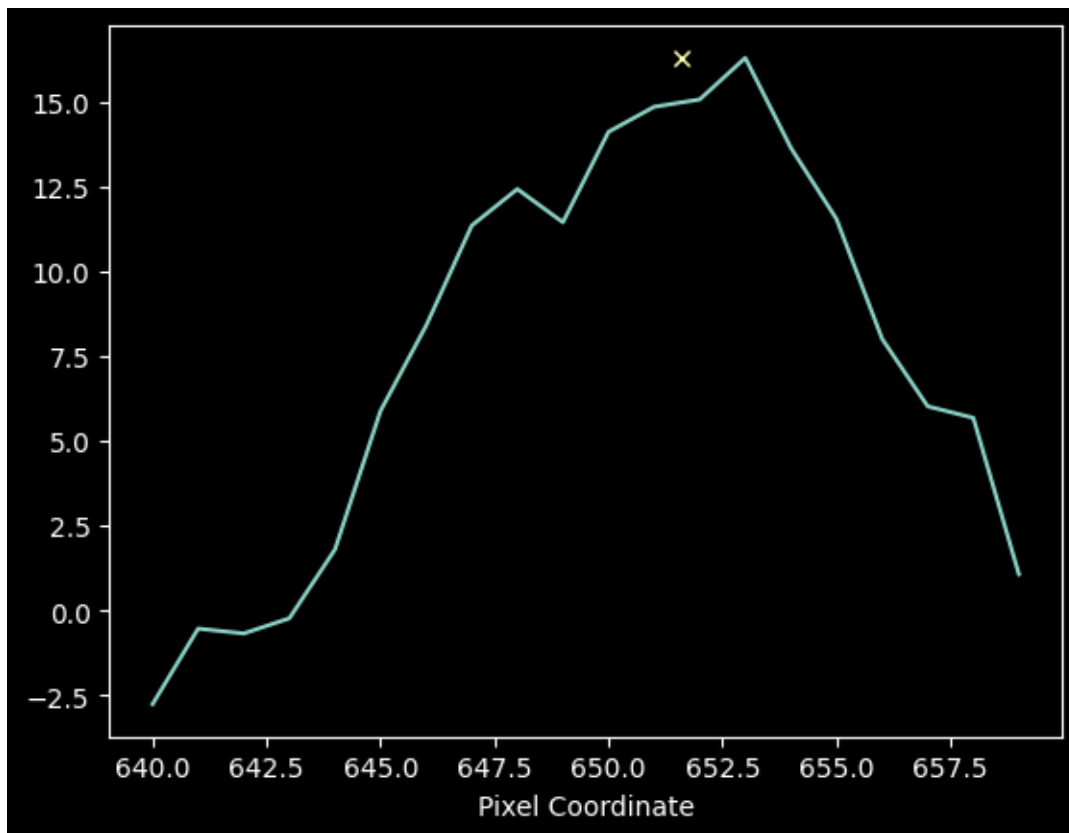
Podemos demostrar que la incertidumbre por línea es insignificante observando una tenue línea de neón:

```
[ ]: # cut out a line and compute its moment 1
cutoutne = ne_spectrum[640:660] - np.median(ne_spectrum)
xcutoutne = xaxis[640:660]
m1ne = (xcutoutne * cutoutne).sum() / cutoutne.sum()
```

```
[ ]: # estimate the per-pixel error in the Neon spectrum
ne_noise_estimate = ne_spectrum[1400:1600].std()
ne_noise_estimate
```

```
[ ]: np.float64(1.0704108932039562)
```

```
[ ]: plt.plot(xcutoutne, cutoutne)
plt.plot(m1ne, cutoutne.max(), 'x')
plt.xlabel("Pixel Coordinate");
```



```
[ ]: # calculate Neon line centroid uncertainty
sigma_m1_left = ((xcutoutne-m1ne)**2 * ne_noise_estimate**2).sum() / cutoutne.
    ↳sum()**2
sigma_m1_right = (xcutoutne.size * ne_noise_estimate**2) *
    ↳((xcutoutne-m1ne)**2*cutoutne**2).sum() / cutoutne.sum()**4
sigma_m1_ne = sigma_m1_left + sigma_m1_right
sigma_m1_ne
```

```
[ ]: np.float64(0.03746963744056958)
```

```
[ ]: print(f"Moment analysis for the faint Neon line yields m1 = {m1ne:0.4f} +/-
    ↳{sigma_m1_ne:0.4f} pixels")
```

Moment analysis for the faint Neon line yields m1 = 651.6098 +/- 0.0375 pixels

Este error es aproximadamente 40 veces mayor, pero aún mucho menor que el ancho de un píxel.

Nuestro mejor ajuste indica que cada píxel mide aproximadamente 0,1 nm (esto se obtiene de la pendiente de la línea ajustada), por lo que nuestra incertidumbre es de 0,0001 nm a 0,004 nm. La dispersión en los residuos es de 0,23 nm, por lo que no se explica por la incertidumbre estadística de nuestro ajuste formal.

```
[ ]: print(f"Standard Deviation of the residuals = {residuals_hgkrnemodel_nedata.  
      ↪std()} nm, "  
      f"Faint Line uncertainty = {sigma_m1_ne * linfit_wlmodel_hgkrne.slope}␣  
      ↪nm")
```

Standard Deviation of the residuals = 0.23808671082730068 nm, Faint Line  
uncertainty = -0.003827022595145164 nm

## 0.5 ¿Qué representa el residuo?

El residuo de nuestro ajuste nos da una idea de la incertidumbre *sistemática* derivada de la combinación de líneas y ajustes imperfectos. Sabemos, gracias a las listas de líneas del NIST, que en algunos casos hay varias líneas que no pudimos distinguir fácilmente en el espectro; esto añade un ligero sesgo a las ubicaciones de los píxeles inferidas.

También puede haber efectos sutiles del propio espectrógrafo que causen pequeñas variaciones sistemáticas en la solución de longitud de onda.

## 0.6 ¿Cómo podríamos mejorar?

Podríamos intentar descombinar las características espectrales individuales ajustando conjuntamente modelos multigaussianos.

Si bien este procedimiento es común, está plagado de errores y, en general, es extremadamente difícil de automatizar.

¡Listo! Ahora aplica esta calibración a los datos y empieza a medir en el Tutorial 3.